

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

PUREXAI: AN AI-ENABLED IOT FRAMEWORK FOR SMART PUBLIC DRINKING WATER QUALITY MONITORING AND PREDICTIVE FILTER MAINTENANCE

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by

BARATH KUMAR E – 23619106

DEVANAND G – 23619109

JAI SURYA S – 23619117

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report " **PUREXAI: AN AI-ENABLED IOT FRAMEWORK FOR SMART PUBLIC DRINKING WATER QUALITY MONITORING AND PREDICTIVE FILTER MAINTENANCE** ", is the Bonafide work of **BARATH KUMAR E – (23619106), DEVANAND G – (23619109), JAI SURYA S – (23619117)** carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya M.E., Ph.D
Associate Professor & Head of
Department of Artificial Intelligence
and Machine Learning
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR

Ms. S. J. Kavitha M.E., (Ph.D)
Assistant Professor
Department of Artificial Intelligence
and Machine Learning
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva - Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Dr. Ishari K. Ganesh**, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Ms. S.J. Kavitha, Assistant Professor, Department of AI & ML**, who has patiently guided us and provided invaluable advice and help throughout the work. We are thankful to our **Project Coordinator, Ms. S.J. Kavitha**. We would like to express our greatest gratitude to our staff members, Librarian, and non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

BARATH KUMAR E – 23619106

DEVANAND G – 23619109

JAISURYA S – 23619117

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF TABLES	iii
	ABBREVIATIONS	iv
1	**INTRODUCTION**	1
	1.1 Overview of the Project	1
	1.2 Scope and Objective	13
2	**LITERATURE REVIEW**	20
	2.1 Literature Overview	20
	2.2 Literature Survey	21
3	**SYSTEM DESIGN**	31
	3.1 Existing System	31
	3.2 Proposed System	32
	3.3 System Architecture	35
	3.4 Hardware Requirements	38
	3.5 Software Requirements	39
4	**IMPLEMENTATION**	41
	4.1 Methodology	41
	4.2 Implementation	49
	4.2.1 Water Quality Data Acquisition Module	49
	4.2.2 Data Preprocessing and Feature Engineering	51
	4.2.3 Filter Health Index Module	52
	4.2.4 Filter Replacement Prediction Module	54
	4.2.5 Water Quality Risk Score Module	57
	4.2.6 Alert and QR System	58
	4.2.7 Dashboard and Monitoring Module	61
	4.3 Hardware Prototype and System Integration	64
	4.4 Integrated System Operation	66
5	**CONCLUSION AND FUTURE WORK**	68
	5.1 Conclusion	68
	5.2 Future Work	72
	5.3 Source Code	77
	5.4 Results	81
	REFERENCES	88

ABSTRACT

Public drinking-water facilities need timely inspection and maintenance decisions, but periodic manual checks can miss changes between visits and fixed filter-replacement schedules may not reflect the actual condition of an individual filter. PUREXAI is an AI-enabled Internet of Things (IoT) framework that addresses this problem by combining an ESP32 sensing layer, wireless data transmission, time-series data storage, machine-learning-based filter assessment, remaining-filter-life estimation, operational risk scoring, local alerts, dashboard monitoring and QR-based tap information. The prototype measures total dissolved solids (TDS), turbidity and water temperature at public-tap nodes using a DFRobot analog TDS sensor, a DFRobot analog turbidity sensor and a DS18B20 temperature sensor. The measurements are timestamped, associated with tap or facility identifiers and transmitted over Wi-Fi so that short-term and historical trends can be examined. The analytical layer uses Random Forest models for filter-state classification and remaining-life estimation. A Filter Health Index (FHI) represents the estimated operational condition of a filter, while the Water Quality Risk Score (WQRS) combines normalized sensor deviation, filter condition and short-term deterioration trends for inspection prioritization. An OLED display and buzzer provide local warnings, and a dashboard and QR-based interface present current values, trends, alerts and sensor status. In the prototype evaluation, three public-tap nodes were sampled once every minute for 10 days, producing 43,200 timestamped observations. The filter-state classifier achieved 97.01% accuracy and a weighted F1-score of 0.968, the remaining-life model produced a mean absolute error of 2.8 days, the operational risk classifier achieved an F1-score of 0.93, and the mean alert latency was 2.1 seconds across 100 alert events. These results indicate technical feasibility under the tested conditions. PUREXAI is a prototype: the FHI bands and risk weights are study-defined, the evaluation covers three taps over 10 days, and the system is not a certification of drinking-water safety or proof of superiority over a controlled baseline.

Keywords

Artificial Intelligence, Internet of Things, ESP32, water-quality monitoring, predictive maintenance, Filter Health Index, Water Quality Risk Score, Random Forest, smart dashboard.

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
Figure 1	PUREXAI system architecture: sensing, edge processing, communication, AI analysis, risk assessment and user interfaces	37
Figure 2	PUREXAI water-quality monitoring interface during prototype operation	51
Figure 3	PUREXAI monitoring and public-information workflow	60
Figure 4	PUREXAI live dashboard showing real-time sensor readings and system status	63
Figure 5	PUREXAI sensor-health interface showing sensor status, latest readings, timestamps and error counts	64
Figure 6	PUREXAI prototype implementation with ESP32 controller, sensors, breadboard circuitry and OLED display	65
Figure 7	Variation of TDS, turbidity and water temperature measurements obtained from the PUREXAI monitoring system during prototype operation	136
Figure 8	Comparison of TDS values at three public drinking-water taps (X, Y and Z) between selected previous studies and the proposed PUREXAI system	136
Figure 9	Reported classification accuracy in selected water-quality studies; values are descriptive and are not a controlled benchmark	137

LIST OF TABLES

TABLE NO	TITLE	PAGE NO.
Table 1	Comparison of Related Studies and PUREXAI	27
Table 2	Comparison of Existing and Proposed System	34
Table 3	Layer-wise Functions of the PUREXAI Architecture	37
Table 4	Hardware Components Used in the Prototype	39
Table 5	Software and System Requirements	40
Table 6	Sensor Configuration Used in Prototype	44
Table 7	Experimental Dataset Configuration and Chronological Split	48
Table 8	Filter Health Index Operating Bands	54
Table 9	Operational Water Quality Risk Score Bands	58
Table 10	Consolidated Evaluation Results of PUREXAI	135

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
IoT	Internet of Things
ESP32	Espressif ESP32 Microcontroller Platform
TDS	Total Dissolved Solids
NTU	Nephelometric Turbidity Unit
FHI	Filter Health Index
WQRS	Water Quality Risk Score
MAE	Mean Absolute Error
RF	Random Forest
OLED	Organic Light-Emitting Diode
LED	Light-Emitting Diode
QR	Quick Response
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
REST	Representational State Transfer
Wi-Fi	Wireless Fidelity
ML	Machine Learning
RUL	Remaining Useful Life
ppm	Parts Per Million
WHO	World Health Organization

CHAPTER 1 – INTRODUCTION

Public drinking-water facilities require regular observation because the condition of the water, the treatment components and the sensing equipment can change over time. The PUREXAI project addresses this operational problem by combining low-cost sensing, wireless communication, machine learning and user-facing monitoring in one integrated framework.

The project focuses on selected indicators rather than a complete laboratory assessment of drinking-water safety. Total dissolved solids (TDS), turbidity and temperature are measured continuously at public-tap nodes, timestamped and retained so that current conditions can be viewed together with short-term and historical trends. The central idea is to move beyond a dashboard that merely displays sensor values: PUREXAI derives a Filter Health Index, estimates remaining filter life, calculates an operational Water Quality Risk Score and generates alerts, thereby connecting sensing with maintenance-oriented decision support.

1.1 Overview of the Project

The PUREXAI framework is an AI-enabled IoT prototype designed for smart public drinking-water quality monitoring and predictive filter maintenance. The embedded layer is built around an ESP32 controller. TDS, turbidity and temperature sensors provide the principal measurements, while Wi-Fi enables communication between the sensing node and the monitoring system.

The architecture separates the system into sensing, edge and communication, data management, AI and decision, and user and alert layers. This separation allows raw observations to move from the physical environment to a structured software representation and then to higher-level indicators that can be interpreted by maintenance personnel.

The project is not limited to one machine-learning model. Its contribution is primarily at the system-integration level: continuous sensing is combined with filter-condition assessment, remaining-life estimation, operational risk classification, local warnings, dashboard monitoring and QR-based public access to tap information.

Background

Water-quality monitoring is best understood as a repeated measurement and interpretation process. A value is useful only when its unit, source, time and measurement context are known. When readings are collected at different times or locations, a consistent record format allows the observations to be compared without losing their identity. PUREXAI uses tap-level identifiers and timestamps to preserve this context and to support later trend

analysis. This is particularly important when a user is reviewing an alert after the original event has passed.

A monitoring installation also needs to distinguish changes in the monitored environment from changes in the measurement system itself. A sudden jump can represent a real event, electrical interference, a loose connection or a sensor that needs maintenance. A time series and sensor-health record do not automatically identify the cause, but they provide evidence that can guide the investigation. A technician can compare the reading with nearby observations, check whether other parameters changed and examine whether a communication or sensor error occurred at the same time.

The project therefore combines numerical measurements with contextual information and derived indicators. The raw values remain important because they are the direct observations, while FHI, remaining-life estimates and WQRS are higher-level interpretations. Keeping these levels, separate helps avoid a common design mistake in which a derived score is displayed without access to the data behind it. The dashboard should allow a user to move from a summary status to the readings and timestamps that contributed to it.

Continuous monitoring should complement established water-quality procedures rather than replace them. The sensors included in PUREXAI are useful for observing selected operational parameters, but they do not cover every hazard that can affect drinking water. The system can help indicate when review is warranted, while confirmatory sampling, laboratory analysis and official response remain separate responsibilities. This boundary is central to the project's intended use and should be maintained in technical documentation and user-facing messages.

Water delivered at public taps depends on treatment and filtration components whose condition changes with use. The measurable properties of the water and the operating state of the filter are related, but the relationship is not visible from a single reading. A technician who sees a change in TDS or turbidity must still decide whether the filter is deteriorating, whether the sensor is faulty, or whether the change is temporary.

Embedded sensing and machine learning offer a way to support this decision. When observations are recorded continuously and associated with a specific tap, the data can be examined for trends, compared with a baseline and used to estimate the condition of the filter. This is the background against which PUREXAI was designed.

Public Drinking-Water Monitoring

A monitoring system for a public facility has to serve more than one type of user. A maintenance technician needs enough detail to investigate a developing fault, a facility

manager needs a concise overview of the taps requiring attention, and a member of the public needs understandable information about the monitored point. PUREXAI separates these needs into technical records and user-facing summaries. The underlying record retains numerical readings, timestamps and the tap identifier, while the interface converts selected outputs into status indicators and alerts. This separation is important because a simple status label is useful for quick decisions but is not sufficient for technical troubleshooting.

The value of continuous observation is temporal context. A single measurement represents one moment and may not reveal whether a condition is stable, gradually changing or temporarily disturbed. A time series allows an operator to compare a current observation with recent readings and to identify a pattern that deserves inspection. In the PUREXAI prototype, the one-minute sampling interval creates a regular sequence of observations. The resulting record can support trend plots, rate-of-change features and a traceable alert history. These capabilities do not guarantee that every abnormal condition will be detected; their effectiveness depends on sensor behavior, data availability, thresholds and the validity of the analytical logic.

Public facilities also create practical constraints that differ from controlled laboratory settings. A device may be exposed to vibration, moisture, electrical interference, accidental disconnection or unstable network access. Sensors may need cleaning or recalibration, and the physical installation must avoid obstructing routine use of the tap. The report therefore treats the prototype as a technical demonstration of an integrated workflow rather than a finished certified appliance. A field-ready version would require a protected enclosure, documented installation procedures, a maintenance schedule and independent verification of the measurements used for operational decisions.

The framework is designed to preserve the distinction between an observed parameter and a conclusion about safety. TDS, turbidity and temperature can provide useful signals for monitoring, but none alone establishes the full microbiological and chemical condition of drinking water. A high or changing sensor reading should be interpreted as a reason to review the monitored point, not as a substitute for a validated sampling and laboratory procedure. The interface should make this limitation clear so that users do not confuse an operational alert with an official declaration that water is safe or unsafe.

Traditional inspection of drinking-water facilities often depends on scheduled checks. Samples may be collected at selected times and analyzed later, meaning that a developing change between inspection points can remain unnoticed. A continuous sensing approach provides a different operating model in which observations are collected at regular intervals and made available to a monitoring platform.

PUREXAI treats the public tap as an operational unit with its own observations and identifiers. Tap-level information is retained rather than being merged into one undifferentiated stream, so a technician can focus attention on the tap or filter whose indicators show deterioration. The framework does not replace certified laboratory analysis; its role is to provide continuous operational information and decision support.

IoT Foundation

The IoT workflow begins when a physical quantity is converted into an electrical signal by a sensor. The ESP32 reads the available sensor outputs, associates the readings with the current sampling cycle and prepares a record for transmission. A successful record must retain the relationship between the measurement, the timestamp and the originating tap. Without this association, historical analysis could combine readings from different places or misinterpret a delayed message as a current observation. Tap identity and time are therefore part of the data model, not merely optional labels shown on the dashboard.

The 60-second sampling interval is a design choice that balances temporal resolution with practical processing and communication demands. A shorter interval would generate more records and might capture faster changes, but it would also increase communication traffic and storage volume. A longer interval would reduce the record count but could miss short-lived variation. The selected interval is the one documented for this prototype; it should not be assumed to be optimal for every water source or sensor. A later deployment should choose its interval after considering the expected rate of change, sensor response time, power availability, network capacity and the action time required by maintenance staff.

Wireless connectivity makes the observations available to a shared monitoring interface, but a connected device must also handle periods when the network is unavailable. A robust design should distinguish between a new valid observation, an old observation still visible on screen and a communication failure. If a network interruption occurs, the interface should show the last successful timestamp rather than presenting stale data as if they were current. Where local memory and firmware support it, records can be buffered and transmitted after reconnection. Such buffering and retry behavior should be documented and tested before being relied on in a production setting.

The edge device has a limited responsibility in the overall architecture. It performs sensor acquisition and basic validation close to the source, while the software and analytical layers provide historical storage, model inference, risk calculation and presentation. This division keeps the physical node relatively compact and allows the monitoring software to evolve without redesigning every sensor connection. At the same time, the interface between

layers must be stable: field names, units, timestamp formats, tap identifiers and error codes should be documented so that the receiving service can validate incoming records consistently.

IoT technology provides the physical and communication foundation for PUREXAI. Sensors convert properties of the water into measurable signals, the ESP32 acquires the readings, and wireless communication transfers the observations to the software system. The prototype samples the three monitored indicators once every 60 seconds. During the reported evaluation, three public-tap nodes operated for 10 days, producing 43,200 timestamped observations.

The value of IoT in this project is therefore not only connectivity. The continuous record provides the temporal context needed by the analytical modules: current readings can be compared with recent behavior, and maintenance decisions can be associated with the condition of a particular tap and filter.

AI Layer

Machine learning is used in PUREXAI to transform prepared observations into outputs that are easier to use for maintenance planning. The project describes two distinct prediction tasks. Filter-state classification assigns an observation to one of the defined filter-condition classes, whereas remaining-useful-life estimation produces a numerical estimate expressed in days. The two tasks should not be conflated: a class describes a broad condition band, while a regression output provides a more specific estimate whose uncertainty may be substantial. The operational risk score is a further derived indicator and is not itself the same as either model output.

A Random Forest is an ensemble of decision trees. During training, the trees are constructed from variations in the training data and candidate features; their outputs are combined to obtain a classification or regression estimate. This approach can model nonlinear relationships without requiring every relationship to be specified as a simple equation. It can also provide feature-importance measures, although those measures should be interpreted carefully when features are correlated or derived from one another. The use of Random Forest in this project is an engineering choice within the integrated framework, not a claim that a new learning algorithm has been invented.

The quality of a model depends on the meaning and quality of its target labels. If filter-state labels are defined directly from the Filter Health Index bands and the FHI is also supplied as an input feature, the model may learn a relationship that is partly encoded by the label-generation rule. This can produce high test metrics without proving that the model independently infers physical filter condition. The report therefore records this as an important

limitation and identifies leakage-controlled evaluation as future work. A stronger experiment would construct targets independently of the predictor features, document the label-generation procedure and test on separate filters or locations.

AI-generated outputs should be displayed with enough context to support a responsible decision. Useful context includes the latest sensor timestamp, the individual measurements, the recent trend, the filter-health band, the risk band and any sensor or network warning. Where possible, a user should be able to inspect the evidence behind a status rather than seeing only a single colored label. The dashboard is a decision-support interface: maintenance personnel remain responsible for follow-up checks, and official drinking-water safety decisions require appropriate external testing and procedures.

Machine learning provides the analytical stage of PUREXAI. The collected sensor measurements are transformed into features that describe current values, short-term variation, rates of change, deviations from a baseline and filter-related information. These features are supplied to Random Forest models for filter-state classification and remaining-life estimation.

The Filter Health Index (FHI) is an operational numerical representation of estimated filter condition; larger values indicate a healthier estimated state. The system groups the index into Healthy, Warning and Critical operating bands, which are defined for the prototype workflow and are not external regulatory thresholds. The remaining-life module treats replacement planning as a regression problem, and the Water Quality Risk Score (WQRS) combines sensor behavior, estimated filter condition and recent deterioration to prioritize inspection.

Problem Statement

The problem addressed by PUREXAI is not simply the absence of a sensor. A sensor-only installation can automate measurement while leaving the operator to interpret each value, search for historical records and decide whether a filter needs inspection. The project addresses the gap between measurement and action by joining continuous observations with an operational filter-health representation, a remaining-life estimate, a risk score and alert mechanisms. The system is intended to make the information flow more consistent and traceable across sensing, analysis and user communication.

The problem can be separated into data, interpretation and response. At the data level, the system must collect readings at a defined interval and preserve source identity. At the interpretation level, it must process noisy or incomplete observations and produce indicators with clearly stated meanings. At the response level, it must present those indicators to the appropriate user and record when an alert is generated. A failure in any one stage can reduce

the usefulness of the whole workflow. For example, a good classifier cannot compensate for incorrect timestamps, and a correct warning is of limited value if it is not displayed clearly or is delivered too late.

The project also addresses the risk of over-interpreting automated outputs. A numerical risk score can help rank observations for inspection, but its value depends on the chosen components, normalization rules and weights. The WQRS weights in PUREXAI are study-defined and should be treated as an operational prioritization scheme rather than a universal standard. Similarly, the FHI operating bands are internal prototype categories. The report uses these labels consistently while avoiding the suggestion that they are official regulatory limits.

A practical solution must make limitations visible as well as features. The monitored indicators are limited to TDS, turbidity and temperature; the reported evaluation covers three public-tap nodes and ten days; and the supplied documentation does not provide a separate numerical calibration-error analysis. These boundaries influence the conclusions that can be drawn. The central problem statement is therefore framed around demonstrating an integrated monitoring and maintenance-support workflow under the reported prototype conditions, rather than proving complete drinking-water safety or universal predictive performance.

Public drinking-water monitoring becomes difficult when measurements are collected only periodically and interpreted manually. A scheduled inspection can miss changes between visits, while a fixed filter replacement schedule may not reflect the actual operating condition of an individual filter or tap. The project addresses the following operational problems:

- Continuous measurements are needed to observe changes between manual inspections.
- Sensor readings alone do not directly indicate filter condition or remaining useful life.
- Maintenance personnel need a way to prioritize taps or filters requiring attention.
- Alerts must reach users quickly enough to support operational response.
- Public-facing information should be associated with the specific tap being monitored.
- Machine-learning outputs must be presented as decision support rather than as a replacement for laboratory or regulatory assessment.

Objectives of the Project

The **first objective** is reliable acquisition. The system should read each available sensor at the configured interval, validate the reading against expected data types and ranges, and preserve the source identifier. Acquisition reliability includes both successful readings and explicit handling of failed reads. A missing or invalid measurement should not silently be converted into a normal-looking value. Instead, the system should preserve a quality indicator

or error record so that the dashboard can distinguish a sensor fault from an observed change in water conditions.

The **second objective** is traceable data management. Each observation should be stored with a timestamp, tap identifier and the monitored parameter values. Traceability supports the reconstruction of trends, comparison between taps and review of the observations surrounding an alert. It also helps investigators determine whether a model output was generated from current data or from an incomplete sequence. A consistent record format makes it easier to test the software and to add further monitoring nodes in later versions.

The **third objective** is maintenance-oriented analysis. The FHI and remaining-life module are intended to represent filter condition in forms that can support inspection and replacement planning. The fourth objective is risk prioritization through the WQRS. These outputs have different roles and should be presented separately: filter health summarizes estimated filter condition, remaining life estimates time until replacement under the model's assumptions, and WQRS combines selected operational signals into a priority band.

The **final objective** is usable communication. Local display and buzzer alerts can notify a person near the installation, while the dashboard supports broader monitoring and the QR interface can expose tap-specific information. Evaluation should cover not only model metrics but also the completeness of records, alert timing, visibility of device health and clarity of the user interface. The current report includes measured model metrics and alert latency; more extensive usability testing and external field validation remain suitable directions for future work.

The objectives of PUREXAI are derived from the implemented prototype and its evaluation. The main objectives are:

1. To collect **TDS, turbidity and temperature** measurements continuously and reliably using an ESP32-based sensing node.
2. To transmit and store the observations with **timestamps and tap identifiers** so that historical trends can be reconstructed.
3. To calculate an operational **Filter Health Index** and estimate **remaining filter life** using Random Forest models.
4. To compute a **Water Quality Risk Score** that supports inspection prioritization.
5. To provide a **dashboard**, local OLED and buzzer alerts, and a **QR-based tap information** interface for different users.

Scope of the Project

The scope of the prototype includes the sensing hardware, embedded acquisition, wireless transfer, time-series record keeping, preprocessing, machine-learning inference, risk scoring, alert generation, dashboard presentation and QR-based access. These components are considered together because the intended value arises from their integration. A disconnected sensor, an isolated model or a dashboard without reliable source data would each demonstrate only part of the workflow. The report therefore describes the system as a chain of connected functions and explains how the output of one stage becomes the input of the next.

The analytical scope is deliberately narrow. The system monitors TDS, turbidity and temperature, and its reported filter-related outputs are based on the available experimental features and labels. It does not claim to measure every chemical contaminant, detect every pathogen or replace laboratory procedures. Additional sensors could be considered in future work, but their addition would require suitable calibration, integration and validation. It would be inappropriate to imply that the current prototype has measured parameters that are not listed in the documented sensor configuration.

The evaluation scope consists of three public-tap nodes sampled once per minute over ten days, producing 43,200 timestamped observations. The chronological division assigns 30,240 observations to training, 6,480 to validation and 6,480 to held-out testing. This split preserves the order of observations and avoids randomly mixing later time points into training. Nevertheless, chronological splitting alone does not resolve every potential leakage concern, especially where features or target labels are derived from the same operational index. Independent sites, filters and laboratory reference data would be required to support broader generalization claims.

The project does not establish commercial readiness. A production deployment would need enclosure and electrical safety review, environmental protection, calibration procedures, maintenance responsibilities, secure device provisioning, network monitoring, backup and recovery procedures, user training and documented escalation steps. These are not reasons to discount the prototype; they identify the additional work required to move from a demonstrator to a dependable public-infrastructure service.

The physical scope includes an ESP32 development board, TDS and turbidity sensors, a DS18B20 temperature sensor, an OLED display, a buzzer driver, wiring, a power supply and a drinking-water or filter assembly. The software scope includes wireless data transmission, time-series storage, analytical processing, a monitoring dashboard, risk and maintenance

indicators, and tap-specific QR access. The AI scope includes Random Forest-based filter-state classification and remaining-life regression, together with the operational WQRS calculation.

The project deliberately limits its claims. The three monitored indicators are useful operational signals, but they are not a complete laboratory test of drinking-water safety. The reported experimental results describe the tested three-tap, 10-day prototype conditions and should not be generalized to all facilities or filter types without further validation.

Significance

The practical significance of PUREXAI is that it organizes several pieces of information into one monitoring workflow. A technician does not need to interpret each reading in isolation: the dashboard can present current measurements, recent behavior, a filter-health category and an operational risk band together. This can make review more systematic, provided the interface keeps the original measurements available and clearly identifies which values are measured and which are derived by software.

The project is also significant as an integration exercise. Embedded sensing, data communication, storage, machine-learning inference and a user interface have different technical requirements. Bringing them together requires consistent data formats, reliable timestamps, clear component boundaries and a strategy for handling missing or invalid observations. The integrated prototype provides a context in which these engineering issues can be discussed and evaluated rather than treating the sensor, model and dashboard as unrelated demonstrations.

Another contribution is the explicit treatment of operational limitations. The FHI and WQRS thresholds are study-defined, and the monitored parameters do not constitute a complete laboratory assessment. Stating these boundaries helps prevent users from treating an automated label as an official certification. In public infrastructure, a transparent limitation statement is part of responsible design because users may make decisions based on the way results are presented, not only on the numerical model output.

The significance of the project should not be overstated. The reported data cover three tap nodes and ten days, and the current documentation does not establish performance across every filter type or location. The value of the work is therefore best described as demonstrating technical feasibility and identifying a clear path for stronger validation. A larger field study can build on this foundation by using independent reference measurements and testing whether the generated alerts improve maintenance decisions.

The significance of PUREXAI lies in the integration of sensing and maintenance decision support. A conventional monitoring interface may tell a technician that TDS or turbidity has

changed; the proposed framework adds a layer that represents filter condition, estimates remaining life and assigns an operational risk category. Instead of treating every observation equally, the system can highlight deteriorating conditions and provide an action-oriented interpretation.

At the prototype stage, the project demonstrates technical feasibility rather than a complete production deployment. The architecture provides a basis for future work involving longer observation periods, geographically diverse facilities, independent laboratory references, stronger leakage-controlled evaluation and controlled comparisons with baseline methods.

System Overview

PUREXAI can be viewed as a sequence of transformations. Water-quality conditions are first observed by sensors. The ESP32 acquires and validates the signals and prepares them for wireless transmission, and the data-management layer records the measurements so that current and historical views can be produced. The AI and decision layer then produces a filter-health interpretation, a remaining-life estimate and the WQRS, which is mapped to an operational band such as Low, Moderate, High or Critical.

The final layer communicates these results to users. The dashboard displays current values, trends, device information and status; the OLED and buzzer provide local warnings; and QR-based tap information makes selected details available at the monitored facility. This end-to-end flow is the central design principle of the project.

Main Contributions

- An integrated IoT prototype for continuous monitoring at public drinking-water facilities.
- Combination of TDS, turbidity and temperature sensing with timestamped, tap-level observations.
- An operational **Filter Health Index** to represent estimated filter condition.
- A **remaining-useful-life** estimation module that supports filter replacement planning.
- A **Water Quality Risk Score** for inspection prioritization based on measured indicators and filter condition.
- Integration of dashboard monitoring, local alerts and QR-based tap information.

The contribution is primarily system integration rather than a new machine-learning algorithm. Evaluation is performed on held-out experimental data so that the reported performance reflects the implemented prototype under the tested conditions.

Applications

One application is inspection prioritization. If multiple taps are monitored, a facility operator may not be able to inspect every location at the same time. A consistent operational risk band can help direct attention to locations whose measurements or estimated filter condition have changed. The band should be interpreted alongside raw readings and sensor-health status, because a high score caused by a failed sensor requires a different response from a high score caused by a persistent measured change. The dashboard should therefore support drill-down from a summarized status to the evidence that produced it.

A second application is filter-maintenance planning. A remaining-life estimate may help organize inspection schedules and replacement inventory, particularly when multiple filters have different operating histories. Such estimates should not be treated as guaranteed replacement dates. Their reliability depends on the quality of maintenance records, the relationship between sensor behavior and actual filter degradation, and the degree to which the evaluated filters resemble the filters in use. Until these relationships are validated, estimates are best used as supplementary planning information.

A third application is operational documentation. Timestamped records and alert histories can help staff review when a condition changed, whether the device was online and which follow-up action was taken. Over time, a structured maintenance record could help compare the frequency of alerts with filter replacement events and identify recurring device problems. The current prototype establishes the concept of this record flow, while more advanced work-order management and long-term maintenance analytics remain future enhancements.

A fourth application is public information through QR access. A tap-specific QR code can direct a user to a page associated with the correct monitored location. The information should be concise, time-stamped and understandable, and should clearly indicate when readings are stale or the sensor is offline. Public-facing information must avoid presenting the prototype's operational status as a formal water-safety certificate. This careful wording is essential because the same system serves both technical users and members of the public who may not know the limits of the measured parameters.

- **Maintenance planning:** a filter can be assigned an operational health state and an estimated remaining life, allowing personnel to identify units that may require inspection.
- **Inspection prioritization:** the risk score combines current sensor deviation with filter condition and recent deterioration.
- **Public information:** each monitored tap can be associated with a QR code that opens tap-specific information.

- **Operational records:** the framework provides a basis for reports, historical trend views and alert histories.

Expected Outcomes

The expected outcome is a functioning prototype that continuously acquires selected water-quality measurements, transmits them through Wi-Fi, stores them for analysis and presents the resulting information through a monitoring interface, with local warnings when defined operational conditions require attention. In the reported evaluation, the filter-state classifier reached 97.01% accuracy with a weighted F1-score of 0.968, the remaining-life model produced an MAE of 2.8 days, and the operational risk classifier achieved an F1-score of 0.93.

These outcomes are interpreted within the limits of the experimental setup. They indicate that the integrated framework can support the intended workflow under the tested conditions; they do not establish drinking-water certification, universal filter-life guarantees, or superiority over an independently controlled baseline.

1.2 Scope and Objective

The scope of a prototype should be specific enough to make its evaluation understandable. PUREXAI focuses on three measured parameters, an embedded controller, wireless data transfer, time-series records and a defined set of maintenance-oriented outputs. The system does not attempt to measure every possible contaminant or replace the laboratory procedures used for formal water-quality assessment. This limited scope allows the report to describe the implemented workflow without implying broader capabilities than the available evidence supports.

The system objectives can be mapped to observable outputs. Acquisition is represented by timestamped readings; traceability is represented by the tap identifier; analytics are represented by filter-state, remaining-life and risk outputs; and communication is represented by the dashboard, local alerts and QR information. A later evaluation can test each objective independently and then assess whether the complete workflow functions as intended. This is more informative than treating the project as successful merely because each component can be demonstrated separately.

Operational success should include clear failure behavior. If a sensor does not return a valid value, the system should not silently present a normal status. If data are old, the dashboard should indicate the age of the last valid observation. If a model cannot produce an output, the user should be told that the result is unavailable rather than shown a stale estimate as if it were current. These requirements support responsible interpretation of the system’s outputs.

The objectives also include documentation. A reproducible project report should identify the hardware, measurement interval, dataset size, split strategy, evaluation metrics and known limitations. Where implementation details such as software versions or exact pin assignments are not supplied, the report should state that limitation rather than invent details. This approach improves technical credibility and provides a clear checklist for future developers who continue the work.

Purpose of Study

The purpose of the study is to examine whether a connected sensing and analytics workflow can provide useful operational information for public drinking-water facilities. The project begins with physical measurements and carries them through communication, storage, preprocessing, model inference, risk scoring and presentation. The study is therefore concerned with both individual components and the way they work together. Its reported evaluation focuses on the prototype configuration and does not claim that the same performance will automatically be achieved in every facility.

A clear purpose statement also defines what counts as a successful prototype. At minimum, the device should collect the selected parameters, preserve timestamps and tap identity, make the records available for analysis and display the resulting indicators. The model outputs should be understandable to the intended users, and alerts should be traceable to the observations that caused them. These objectives provide a practical framework for discussing the results without assuming that every aspect of field deployment has already been solved.

The study uses the reported 10-day, three-node dataset to evaluate the integrated workflow. The one-minute sampling interval creates a structured time series that supports chronological division into training, validation and held-out testing data. This arrangement provides a reproducible basis for the reported metrics, although the quality of any machine-learning conclusion also depends on the target labels, preprocessing rules and independence of the held-out examples.

The purpose is not to replace established water testing or maintenance procedures. The system is intended to make selected measurements and operational indicators easier to observe and review. If an indicator suggests a problem, the appropriate next step is an inspection or confirmatory procedure, not an unsupported assumption that the dashboard alone establishes water safety. This boundary informs the scope, evaluation and recommendations throughout the report.

The purpose of this study is to design, implement and evaluate an AI-enabled IoT prototype that connects physical water-quality sensing with maintenance-oriented decision support at

public drinking-water facilities. The study examines whether an integrated workflow, from sensor to dashboard, alert and QR-based public information, can operate technically under the tested conditions.

General Objective

The general objective is to connect sensing, communication, data management, AI analysis and user interfaces into a single operational workflow. The sensing layer provides measurements; the communication layer makes the observations available; the data-management layer retains them; the AI layer interprets filter condition and remaining life; and the user layer presents the results to maintenance personnel and public users.

Specific Objectives

1. Acquire TDS, turbidity and temperature at a 60-second sampling interval from each tap node.
2. Associate every observation with a timestamp and a tap or facility identifier.
3. Clean, scale and transform observations into features describing current values, trends and filter context.
4. Classify filter state using Random Forest and estimate remaining filter life using Random Forest regression.
5. Calculate the WQRS from sensor deviation, inverse FHI and short-term deterioration, and map it to operational risk bands.
6. Provide local (OLED, buzzer) and platform alerts and measure alert latency.
7. Provide a dashboard and tap-specific QR access, and report evaluation measures on held-out data.

Functional Scope

The functional scope covers continuous sensing of TDS, turbidity and temperature; edge acquisition through the ESP32; wireless transmission; time-series storage; machine-learning-based analysis; risk scoring; alerting; dashboard access; tap-specific QR information; and local status display through an OLED and buzzer.

Technical Scope

The technical scope includes embedded measurement, wireless communication, structured records, preprocessing, feature generation, classification, regression, risk calculation, alert handling and user-interface presentation. These tasks have dependencies. The model layer relies on consistent input features, while the dashboard relies on correctly associated outputs. A technical fault in an early stage can affect every later stage, so the system should preserve

enough metadata to identify where a record originated and whether it passed the relevant validation checks.

The report describes HTTP/REST communication over Wi-Fi as the data-transfer path. In a complete implementation specification, the request structure should identify required fields, data types, units and error responses. The receiving service should validate the record before accepting it into storage. This does not require the embedded device to implement every analytical task; it requires a clear boundary so that each component knows what information it receives and what responsibility it has.

The machine-learning scope consists of Random Forest classification for filter state and Random Forest regression for remaining life. The operational WQRS combines normalized sensor deviation, inverse FHI and short-term deterioration. The models and formula have different purposes, so their outputs should be labeled distinctly in the interface. A classification category is not a remaining-life estimate, and neither should be confused with a direct measurement of water quality.

The technical scope also includes monitoring the state of the sensing and communication process. A system that reports a numerical value without indicating when it was measured can mislead the user if the device has gone offline. A useful record and interface should therefore preserve freshness information and identify missing or invalid observations. These requirements support the reliability of the complete workflow rather than adding a separate prediction feature.

The technical scope includes the embedded acquisition program, the Wi-Fi communication path (HTTP/REST API), time-series records, preprocessing and feature engineering, Random Forest classification and regression, the WQRS calculation, and the dashboard and sensor-health interfaces. A chronological 70%/15%/15% split of the 43,200 observations is used for training, validation and held-out testing.

Application Scope

The architecture can support monitoring at individual taps, treatment units or similar facility locations, provided that the sensing configuration is appropriate. The reported evaluation, however, covers only three public-tap nodes over 10 days.

Importance of the Project

Public infrastructure benefits when maintenance is guided by observed condition rather than by a fixed schedule alone. By representing filter condition and risk numerically, PUREXAI

helps personnel decide which tap or filter should receive attention first, and the tap-level record makes those decisions traceable.

System Functionality Overview

In operation, sensors produce observations, the ESP32 acquires and transmits them, the software stores the records and the analytical layer processes the relevant features. FHI, remaining life and WQRS are updated from the available observations, the dashboard presents current and historical information, and local and platform-level alerts are generated when the operational condition requires attention.

Methodology Overview

The methodology proceeds from requirement analysis and system design through sensor integration, data acquisition, transmission, storage, preprocessing, feature engineering, model training, evaluation, risk scoring, alert generation, dashboard development, QR integration and testing. Each step is described in Chapter 4.

Hardware and Software Requirements

The software can be considered as a set of responsibilities rather than a single application. The embedded acquisition program reads the sensors and transmits records. The communication service receives and validates records. The storage component retains timestamped observations. The preprocessing layer handles invalid or missing data and derives features. The model layer performs classification and regression. The risk engine computes WQRS and maps it to a defined operational band. The dashboard displays live and historical information, while the QR interface exposes selected tap-specific details. This functional separation supports modular testing and makes responsibilities easier to assign during maintenance.

A complete software inventory would normally include the programming languages, framework versions, library dependencies, model artifact format, database technology, operating environment and deployment configuration. The available report describes software functions but does not provide a complete versioned dependency list. Therefore, this expanded documentation does not invent specific framework names or version numbers. The final implementation record should be updated from the actual repository and environment files so another developer can reproduce the build without guessing which dependencies were used.

Input validation is important at the software boundary. The service should verify that required fields exist, values can be parsed, timestamps use a consistent format and tap identifiers are recognized. Numeric checks should distinguish a physically implausible reading from a legitimate measurement near an operating boundary. Rejected records should create diagnostic

information without exposing internal details to public users. The dashboard should present sensor faults in a way that supports action, such as asking a technician to inspect the device, instead of silently substituting a default value.

The dashboard and QR page should also be designed with different access needs in mind. Maintenance users may need historical plots, error counts, alert history and model outputs. Public users may need a simplified tap identifier, last update time and a clear explanation of what the monitored indicators do and do not mean. If the same service serves both audiences, access rules should prevent private operational controls from being exposed on the public page. The report describes the interfaces conceptually; the exact authorization and deployment mechanisms should be verified from the implemented software before being claimed as completed features.

The hardware consists of an ESP-WROOM-32 controller, DFRobot Gravity analog TDS and turbidity sensors, a DFRobot Gravity waterproof DS18B20 temperature sensor kit, a 0.96-inch OLED display, a buzzer driver, LEDs, resistors, a breadboard, jumper wires and a power supply. The software requirements are described by function: embedded acquisition, wireless communication, time-series storage, machine-learning engine, risk engine, dashboard and QR interface. Both are detailed in Chapter 3.

Benefits

- Continuous observation between manual inspections.
- Tap-level traceability of measurements and decisions.
- Maintenance support through FHI, remaining-life estimation and WQRS.
- Immediate local warnings and platform alerts.
- Public access to tap-specific information through QR codes.

Challenges and Limitations

The monitored indicators do not constitute a complete laboratory analysis, the FHI thresholds and WQRS weights are study-defined operating values, and the reported model performance is specific to the tested data and configuration. The FHI appears among the recorded feature fields while the operating classes are defined from FHI bands, so the filter-state result requires careful interpretation. The supplied experimental record does not report a separate numerical calibration-error analysis.

Future Enhancements

A future version could provide a clearer lifecycle for each monitored node. The lifecycle would begin with installation and calibration, continue through routine monitoring and alert review,

and include maintenance events such as sensor cleaning, recalibration or replacement. Recording these events alongside the sensor history would make the dataset more informative and would help distinguish changes in water conditions from changes caused by hardware service.

The dashboard could be extended with report exports that summarize selected time periods, missing-data intervals, alerts and maintenance events. Each report should identify the tap, date range, latest observation time and any limitations that affect interpretation. A report that includes only a risk label without its timestamp or data-quality context may be misleading when read later. Exported summaries should therefore preserve enough context to support a technical review.

An additional improvement would be a formal alert acknowledgement workflow. Maintenance staff could mark an alert as reviewed, record the inspection outcome and indicate whether follow-up is required. This would create a feedback loop between the monitoring system and real operational action. Such a workflow would need appropriate access control and a clear audit trail; it should be treated as a future feature unless it is already present in the actual software.

Finally, future development should prioritize validation before adding complexity. More sensors, more model types and more dashboard features do not automatically make the system more dependable. The highest-value next steps are independent reference measurements, leakage-controlled model evaluation, longer operation, transparent alert testing and documentation of calibration and maintenance. These improvements would provide a stronger basis for judging whether additional technical features are warranted.

Future work includes longer-term and multi-location testing, independent laboratory reference measurements, leakage-controlled model evaluation, additional sensors and a more robust production enclosure (Section 5.2).

Expected Outcomes of the Study

The study is expected to deliver a working prototype, a documented analytical workflow and a quantitative evaluation that together demonstrate technical feasibility within clearly stated limits.

CHAPTER 2 – LITERATURE REVIEW

The literature direction behind PUREXAI is the convergence of IoT-based sensing and machine-learning-based analysis for water-quality monitoring. The project is positioned within work on continuous sensing, water-quality prediction, anomaly detection, embedded IoT, resource management and predictive maintenance. Earlier monitoring systems often focus on collecting measurements and presenting them to an operator; more recent work adds analytical methods that can recognize patterns or classify conditions. PUREXAI builds on this direction but concentrates on system integration, combining sensor monitoring with filter-condition assessment, remaining-life estimation, operational risk scoring and alerts.

2.1 Literature Overview

The reviewed research can be organized into four related strands. The first strand concerns sensing and IoT architectures that connect physical measurements to networked data systems. The second concerns machine-learning methods for water-quality prediction, anomaly detection and classification. The third concerns embedded and edge systems that must operate under practical communication and hardware constraints. The fourth concerns predictive maintenance and operational decision support, where measured condition is used to help plan inspection or replacement. PUREXAI sits at the intersection of these strands and uses their combined rationale to organize its system architecture.

IoT-oriented monitoring research emphasizes the value of collecting measurements at regular intervals and transmitting them to a centralized interface. Such systems can reduce dependence on manual data collection and make measurements available for trend analysis. However, connectivity by itself does not ensure that the resulting information is reliable or actionable. Sensor calibration, timestamp consistency, communication loss and user interpretation remain important concerns. PUREXAI reflects this broader view by including data preprocessing, sensor-health information, risk scoring and local alert mechanisms in addition to basic sensor acquisition.

Machine-learning research contributes methods for identifying patterns in multivariate data. Classification can group observations into operational categories, while regression can estimate a continuous target such as remaining life. These methods require representative training data, clear labels and an evaluation procedure that reflects the intended deployment. An algorithm may perform well on a test partition yet generalize poorly when the sensor type, filter, location

or environmental conditions change. This distinction is especially important for infrastructure systems, where a model output may influence a maintenance decision.

The literature also highlights the difference between prediction and assurance. A prediction is an estimate based on data and model assumptions; assurance requires a validated process, appropriate reference measurements and a clear definition of acceptable performance. PUREXAI's current results demonstrate performance within the described prototype dataset, but they do not certify drinking-water safety or prove that the model will behave similarly at every site. The literature review therefore motivates both the integrated design and the conservative interpretation of the reported results.

IoT-based water-quality monitoring reduces some of the limitations of periodic manual sampling. Sensors can be connected to an embedded controller, readings can be collected at regular intervals, and measurements can be transmitted to a remote monitoring platform. This approach is particularly relevant to public facilities where small sensing nodes can be placed near individual taps or treatment units. Several reviews of machine learning and IoT for water-quality monitoring and prediction are available [2], [3], [4], [5], [8], [11].

Parameters such as TDS, turbidity and temperature are widely used to describe water conditions, and these indicators are also used in PUREXAI. The difference is that the project does not stop at displaying the measurements; it uses the resulting time series as input to analytical and maintenance-oriented functions. Formal assessment of drinking-water quality is governed by guideline documents such as the WHO guidelines [6], which PUREXAI does not replace.

Continuous measurement also introduces practical concerns. Multiple nodes may generate frequent observations, communication resources may be limited, and alerts must arrive in time to be useful. A complete system must therefore balance sensing frequency, processing, communication, storage and user access.

2.2 Literature Survey

IoT-Based Water-Quality Monitoring and Sensor-Based Observation

Sensor-based monitoring systems convert physical measurements into digital records that can be stored, visualized and analyzed. A basic system generally includes a sensor, an acquisition controller, a communication path and a display or storage service. More complete systems also include data validation, device-health monitoring and mechanisms for notifying users when measurements change. This layered view is useful because each component has its own failure

modes. A sensor can drift, a controller can reset, a network can drop packets and a dashboard can display outdated records if freshness is not checked.

The three indicators selected for PUREXAI provide complementary operational information. TDS is associated with the concentration of dissolved substances represented by the sensor's measurement method. Turbidity indicates optical cloudiness and can respond to suspended particles. Temperature provides context for the measured water and can influence how some sensors behave. These indicators should be retained separately instead of being collapsed into one raw value. The system may later derive a combined score, but keeping the original measurements available allows users to understand which signals changed and helps investigators troubleshoot the source of an alert.

Sampling frequency is a system parameter rather than a universal constant. A fixed interval simplifies the organization of the time series and makes missing observations easier to identify. It also creates a predictable record volume that can be estimated before deployment. At one observation per minute, each continuously operating node produces 1,440 sampling opportunities per day. Across three nodes and ten days, the expected total is 43,200 observations, matching the reported dataset configuration. This arithmetic describes the sampling design; it does not imply that every individual sensor reading is perfectly accurate or that no data-quality issues occurred.

For a deployed system, the sensing stage should record more than numeric values where practical. Device identity, firmware version, sensor status, acquisition time, transmission time and a quality flag can help separate environmental changes from technical faults. The current report documents timestamps, tap identifiers and sensor-health information. Future versions could formalize the complete metadata schema and include calibration history, last maintenance time and explicit missing-data reasons. These additions would improve auditability without changing the central architecture of the project.

Sensor-based monitoring converts physical properties of water into measurable signals that can be logged at a fixed interval. Reviews in this area describe how IoT nodes, wireless links and data platforms are combined to support water-quality assessment [2], [5], [8]. PUREXAI follows this model, using TDS, turbidity and temperature sensors connected to an ESP32 and sampled once every 60 seconds.

ESP32 and Embedded Monitoring

The ESP32 serves as the embedded controller in the PUREXAI prototype because it provides a compact platform for sensor interfacing and wireless communication. The controller coordinates the acquisition cycle and transfers observations to the monitoring software. Its role

is important at the boundary between the physical environment and the software system: analog and digital sensor outputs must be read consistently, and each record must be prepared in a format that the receiving service understands. The report identifies the ESP-WROOM-32 controller and the listed sensor components as the hardware foundation of the prototype.

Embedded firmware should be designed as a repeatable cycle rather than a collection of unrelated sensor reads. At the beginning of a cycle, the device can verify that the required sensors are available, acquire each measurement, apply basic plausibility checks, attach the time and tap identifier, and attempt transmission. If one sensor fails, the system should retain the status of the other readings while marking the failed channel appropriately. A single failed sensor should not silently cause all parameters to be reported as valid, nor should a default numeric value be interpreted as a genuine observation.

Wireless transmission introduces asynchronous behavior. The embedded controller may obtain a reading at one time, but the monitoring service may receive it later because of network delays or retries. Keeping acquisition timestamps distinct from server-receipt timestamps is a useful design practice for future versions. It allows operators to distinguish a genuinely recent reading from an old record that has only just arrived. The current documentation describes timestamped observations and Wi-Fi communication; a production implementation should specify the clock source, time synchronization procedure and behavior when the network is unavailable.

Power, wiring and enclosure design are also part of embedded reliability. Sensor connections must remain stable, analog signals should be routed to reduce avoidable interference, and the power supply must support the controller and peripherals. A breadboard arrangement is appropriate for a prototype but may be vulnerable to movement or accidental disconnection in a public setting. Before field deployment, the assembly should be transferred to a protected and maintainable enclosure with clear connector labeling, strain relief, suitable power protection and a documented inspection procedure.

The ESP32 is suitable for the project because one compact controller can connect to sensors, perform basic processing and provide wireless communication. Espressif technical documentation and the product reference serve as the hardware foundation [1], [7]. In PUREXAI the ESP32 performs data acquisition and communication, keeping the physical interface close to the monitored facility. The embedded approach is important because the project is intended as a practical prototype rather than a purely software-based simulation.

Machine Learning for Water Quality and Anomaly Detection

Water-quality time series can contain both normal variation and events that deserve attention. A machine-learning model can learn relationships among features when the training dataset

contains examples that adequately represent the target conditions. For classification, the target is a category; for regression, the target is a numerical value. The usefulness of either model depends on whether the target reflects a meaningful real-world state and whether the evaluation data represent conditions likely to occur after deployment. Model complexity cannot compensate for ambiguous labels or unrepresentative observations.

Random Forest is suitable as a baseline model for structured sensor features because it can capture nonlinear decision boundaries and interactions without requiring a linear relationship between every input and target. Classification metrics should include more than accuracy when classes are imbalanced. Weighted precision summarizes the reliability of predicted classes while accounting for class support; weighted recall measures the fraction of true examples recovered under the same weighting; weighted F1 combines precision and recall. The reported filter-state metrics provide a fuller view than accuracy alone, but they still depend on the validity of the class labels and split procedure.

Anomaly detection is related to, but different from, supervised classification. A supervised classifier learns from examples assigned to known classes. An anomaly-oriented method may instead flag observations that differ substantially from an established baseline or learned normal pattern. PUREXAI's risk score includes normalized sensor deviation and a short-term deterioration component, making it a rule-based operational prioritization mechanism rather than evidence of a separate unsupervised anomaly model. The report should preserve this distinction so readers understand which functions are documented and which would be future extensions.

Model evaluation should be reproducible. A complete experimental record would state the feature list, label definitions, model settings, random seeds where relevant, software versions, training/validation/test boundaries and any preprocessing parameters learned from the data. Scaling parameters should be fitted on training data and then applied unchanged to validation and test data to avoid using information from the evaluation period. The available report specifies a chronological 70/15/15 split, but it does not provide every item of a full reproducibility package. These details should be added when the original training scripts and configuration are available, rather than guessed.

Reviews of machine learning and IoT for water-quality monitoring discuss methods that extend monitoring beyond visualization by supporting prediction, classification and anomaly detection [2], [3], [4]. Different models can learn relationships among measured parameters and observed operating conditions.

PUREXAI adopts Random Forest for two principal tasks: filter-state classification and remaining-life estimation. The project also derives a risk score from sensor deviation, filter condition and short-term deterioration. The choice reflects the system-integration objective; Random Forest is not presented as a novel algorithm but as one analytical component within a larger IoT workflow.

Drinking-Water Management

Work on machine learning in drinking-water quality management and distribution systems includes prediction and anomaly detection based on historical and continuously collected measurements [9]. Such studies show why time-series information is useful for identifying unusual operating conditions. However, predicting water quality alone does not indicate whether the filter responsible for treatment is approaching the end of its useful operation. PUREXAI addresses this additional need through the Filter Health Index and the remaining-life module.

Embedded IoT and Machine Learning

A hybrid machine-learning and embedded-IoT approach combines sensing, local processing and intelligent analysis [10]. This integration is relevant to PUREXAI because public drinking-water facilities benefit when measurements can be transmitted and analyzed without depending entirely on manual inspection. Real-world deployment, however, introduces communication reliability, limited computing resources and the need to deliver information quickly; these concerns are reflected in the compact edge controller, regular sampling, wireless transmission, dashboard and local warning mechanisms of PUREXAI.

Resource Management

Resource management in an IoT monitoring system includes the number of sensor reads, the amount of transmitted data, storage growth and the processing required to calculate indicators. At a fixed sampling interval, the record volume can be estimated before deployment. The documented configuration of three nodes, 60-second sampling and ten days produces 43,200 timestamped observations. If the number of nodes or monitoring duration increases, storage and communication needs will grow proportionally unless the record format or sampling plan changes.

A fixed sampling interval simplifies system behavior because each node is expected to report at predictable times. This makes gaps easier to identify and supports straightforward time-series plotting. However, a fixed interval may not be optimal for all events. A future system could evaluate whether adaptive sampling is useful, but any change would need to preserve

comparability and be justified by sensor response time, network availability and the operational speed at which a change must be detected.

Processing can be divided between the embedded node and the monitoring software. The ESP32 handles acquisition and basic validation, while the software side performs storage and the documented analytical functions. This arrangement avoids requiring every model calculation to run on the microcontroller. If future versions move more processing to the edge, the design should consider available memory, compute time, model size, update procedures and whether the local device can retain reliable operation during network outages.

The measured mean alert latency of 2.1 seconds across 100 events is one indicator of responsiveness, but it does not describe total resource efficiency. A broader study could record communication volume, storage growth, processor load and recovery behavior. These measurements would help decide whether the current architecture is suitable for a larger number of public taps and whether any optimization is necessary before deployment.

Continuous IoT monitoring can create resource-management challenges because several sensing nodes may operate simultaneously and send readings to a common platform; resource-allocation optimization for IoT-enabled water-quality monitoring has been studied [12]. These considerations support the use of a defined sampling interval and an organized communication pipeline. PUREXAI samples at 60-second intervals, and the measured mean alert latency of 2.1 seconds across 100 alert events provides one operational indicator for the completed prototype.

Evaluation and Validation

Recent reviews emphasize data availability, validation, monitoring coverage and real-world deployment [3], [11]. A single accuracy value may not fully describe how an intelligent monitoring system behaves in practice, so PUREXAI reports several measures: accuracy and weighted F1-score for filter-state classification, mean absolute error for remaining life, F1-score for the risk classifier and measured alert latency. The data are divided chronologically into training, validation and held-out test observations, which is more representative of sequential monitoring than a randomly mixed split.

Predictive Maintenance

Predictive maintenance provides a practical reason to combine sensor data with machine learning. Instead of replacing every filter according to the same fixed schedule, a system can monitor the current condition and indicate when inspection or replacement may be needed. Operating time, sensor trends, water-quality readings and maintenance history are possible

inputs to maintenance models. PUREXAI uses the available experimental features to represent filter condition and estimate remaining operating life, so that a unit with a lower health state or higher operational risk can be reviewed before a unit whose indicators remain stable.

Comparison of Related Work

Table 1 summarizes the related studies cited in this report and the aspect of water-quality monitoring each addresses. The entries are based on the titles and the descriptions given in the cited sources; reported accuracy values are compared in Section 5.4.

Table 1 : Comparison of Related Studies and PUREXAI

Ref.	Study focus	Contribution area	Relation to PUREXAI
[2]	Machine learning and IoT for water-quality monitoring (review)	Overview of methods and IoT-based monitoring	Background for sensing and ML analysis
[3]	ML, deep learning and IoT for water-quality prediction (systematic review)	Prediction approaches	Motivates ML-based analysis
[4]	Role of IoT, AI and ML in smart water management	Smart water management	Supports AI-assisted monitoring
[8]	Water-quality analysis using IoT and ML (critical review)	IoT and ML models	Background for sensing and ML
[9]	ML in drinking-water quality management (distribution systems)	Drinking-water management	PUREXAI adds filter-level assessment
[10]	Hybrid ML and embedded IoT water-quality monitoring	Embedded IoT with ML	Closest embedded-IoT precedent
[12]	Resource allocation in IoT-enabled water-quality monitoring	Resource management	Supports defined sampling and pipeline
[13]	WaQuPs: ensemble learning for water-quality prediction	Water-quality prediction	Accuracy compared in Section 5.4
[14]	Pond water-quality metrics with IoT sensors and ML	IoT sensing with ML	Accuracy compared in Section 5.4
[15]	IoT sensors with a probabilistic ML model	Water-quality level estimation	Accuracy compared in Section 5.4
[16]	AquaBot: smart pond monitoring system	Integrated monitoring	Accuracy compared in Section 5.4
Proposed work	PUREXAI	Sensing, FHI, remaining life, WQRS, alerts, dashboard, QR	Integrated maintenance-oriented workflow

Research Gap and Need for Integrated Systems

The research gap identified for PUREXAI concerns the integration of multiple operational functions. Sensor monitoring can produce a stream of readings, predictive models can estimate a target, and a dashboard can present information, but a maintenance-oriented workflow must connect these parts reliably. It should preserve the origin and time of each observation, expose device faults, interpret model outputs, prioritize follow-up and make the result accessible to the intended user. The project combines these functions in one prototype and documents the links between them.

A second gap concerns filter maintenance. Water-quality monitoring and filter condition are related, but they are not identical. A water measurement describes an observed property of the sample, whereas a filter-health estimate attempts to summarize the condition of a treatment component. A remaining-life estimate adds a time-oriented prediction. A complete validation would need independent information about filter condition and replacement events so that the model could be evaluated against a meaningful reference. The current prototype provides an integrated workflow and reported regression metric, while independent filter-level validation remains a limitation and a future research priority.

A third gap concerns operational communication. A monitoring platform may be technically capable yet fail to support action if alerts are ambiguous, delayed or detached from the relevant tap. PUREXAI combines dashboard presentation with local OLED and buzzer warnings and QR-based tap information. The measured mean alert latency of 2.1 seconds across 100 alert events is one indicator of the tested workflow's responsiveness. It does not measure every aspect of usability, delivery reliability or user response, which should be studied separately in a broader deployment.

The final gap concerns responsible interpretation. AI-enabled infrastructure must communicate uncertainty, data freshness and the difference between an operational warning and a formal assessment. The current report explicitly identifies its study-defined FHI bands and WQRS weights and notes that the three monitored parameters do not constitute a complete laboratory assessment. This framing is a strength of the documentation because it sets a boundary around the claims and points toward the external validation required before the system could be relied on for higher-stakes decisions.

Many existing systems focus primarily on sensing, water-quality prediction, IoT connectivity or resource management. These functions are valuable individually, but they do not necessarily provide a complete workflow from physical measurement to maintenance action. The following gaps emerge from the review:

Existing approaches can be understood along a spectrum. At one end, manual monitoring relies on scheduled sampling, operator records and later interpretation. This approach can use established laboratory procedures, but it may provide limited temporal coverage between sampling events. At another point on the spectrum, an electronic sensor may continuously measure a parameter and display the current value. This reduces manual collection effort but may still require an operator to determine what a change means. A networked monitoring system can add historical charts and remote access, yet it may not include maintenance prediction or a consistent alert-prioritization method.

The limitations of an existing system should be stated carefully. Not every manual or sensor-based system lacks alerts, historical data or automated analysis; advanced systems may already provide these functions. The comparison in this report therefore describes common limitations of basic or disconnected monitoring workflows rather than asserting that all existing systems share them. PUREXAI's proposed value lies in the combination of the documented features within one prototype and in the traceability of its end-to-end data flow.

A useful comparison considers data continuity, source identification, interpretation, maintenance support, alerting, public access and validation. Data continuity asks whether readings are collected at a defined interval. Source identification asks whether each record belongs to a specific tap. Interpretation asks whether users see raw values only or also derived indicators. Maintenance support asks whether filter condition and estimated remaining life are available. Alerting asks whether a local and remote user can notice a condition. Validation asks whether model performance and sensor behavior have been examined under documented conditions.

The existing-system analysis also provides a baseline for future evaluation. To establish that PUREXAI improves on a simpler system, a controlled comparison would need to define the baseline and compare equivalent data, tasks and operating conditions. The current report does not claim such a controlled superiority study. It demonstrates integration and reports prototype metrics. Future work can compare manual inspection intervals, threshold-only alerts and the proposed risk-score workflow using pre-defined criteria such as detection delay, false-alert frequency, maintenance workload and time to investigate an event.

1. Limited integration of continuous sensing with **filter-condition assessment** and **remaining-life estimation**.
2. Limited **operational risk scoring** that combines sensor deviation, filter health and deterioration trends.
3. Limited combination of **local alerts, dashboards and tap-level public information** in one framework.

4. Need for **careful validation** and honest interpretation of AI outputs in public infrastructure.

PUREXAI addresses this gap at the system-integration level. It is positioned as an integrated prototype rather than a claim of a new machine-learning algorithm.

Summary of Literature Findings

The literature establishes the rationale for combining IoT sensing with machine-learning analysis. Continuous monitoring provides more temporal information than periodic sampling, while machine learning can transform observations into classifications, predictions and anomaly-oriented indicators. The reviewed direction also highlights constraints involving communication, resource management, validation and real-world deployment, which influence the architecture of PUREXAI: a compact ESP32 node, structured data storage, chronological evaluation and explicit caution around the interpretation of AI-generated outputs. The next chapter converts these observations into a system design.

CHAPTER 3 – SYSTEM DESIGN

The PUREXAI system design is based on a layered architecture. The sensing layer observes TDS, turbidity and temperature; the edge and communication layer uses the ESP32 to acquire and transmit the measurements; the data-management layer stores timestamped observations; the AI and decision layer generates filter-health, remaining-life and risk indicators; and the user and alert layer presents the results through dashboards and local warnings. This organization separates physical measurement from analytical interpretation while preserving the data path between them.

3.1 Existing System

Overview

Existing water-quality monitoring approaches commonly collect measurements at selected intervals, or continuously, and display them to an operator. Traditional manual inspection may involve sampling and later analysis, while IoT systems can automate measurement and transmission. Such systems provide useful observations but may leave interpretation to the operator: a technician may see a change in TDS or turbidity without having a direct numerical representation of filter condition or remaining filter life.

Limitations of Existing Systems

1. **Periodic/manual inspection:** a developing change can occur between scheduled visits.
2. **Lack of continuous data:** samples analyzed later do not show how conditions evolve between visits.
3. **Lack of predictive maintenance:** raw measurements do not directly answer whether a filter is approaching replacement, and a fixed schedule may not reflect the actual state of an individual tap.
4. **Lack of integrated alerts:** a dashboard without local alerts may be slow to act on, while local warnings without historical storage cannot support trend analysis.
5. **Lack of tap-specific information:** measurements are not always associated with an individual tap or made available to the public.
6. **Limitations of sensor-only dashboards:** they display values but do not interpret filter condition or prioritize inspection.

The PUREXAI design starts from this existing monitoring model but adds AI-based decision support and operational risk scoring. The objective is not to discard sensing or dashboards but to connect them with maintenance information.

3.2 Proposed System

The proposed system uses a modular design so that the sensing, data-management, analytics and presentation functions can be understood and tested independently. The sensor layer is responsible for observations; the embedded layer acquires and transmits them; the data layer retains records; the analytical layer produces FHI, remaining-life and WQRS outputs; and the interface layer communicates status and alerts. Modularity helps with troubleshooting because a failure can be traced to a stage instead of being treated as one undifferentiated system problem.

The proposed workflow begins with a sampling event and ends with a user-visible result. During acquisition, the controller reads the sensors and prepares a timestamped record. During transmission, the record is sent to the monitoring software. During preprocessing, invalid or incomplete data are handled according to the documented rules, and derived features are calculated. The model layer generates the relevant predictions, after which the risk engine combines selected inputs and maps the result to an operational band. Finally, the dashboard and local indicators present the status and the system records the event for later review.

Each stage should have a clear contract. For example, the acquisition stage should define parameter names, units and quality flags; the communication stage should define the request format and error response; the storage stage should define how observations are indexed by time and tap; and the model stage should define the feature order and expected input range. Clear contracts reduce integration errors and make it easier to replace a component without silently changing the meaning of data. These contracts are design recommendations consistent with the layered architecture; exact implementation details should be confirmed against the project source code where available.

The proposed system also distinguishes local action from remote monitoring. A local OLED and buzzer can provide an immediate indication near the tap even when no one is looking at the dashboard. The dashboard, in turn, supports broader supervision, historical review and sensor-health inspection. QR access is a separate path intended for tap-specific information. The same underlying observation can therefore serve different audiences, but each interface should use language appropriate to its user and should display the time of the last valid measurement.

Overview

The proposed PUREXAI framework integrates physical sensors, an ESP32 edge node, Wi-Fi communication, data storage, Random Forest models, a Water Quality Risk Score, local alerts, a dashboard and QR-based tap information. The workflow begins with continuous acquisition

of TDS, turbidity and temperature. Each observation is timestamped and associated with a tap or facility identifier. After preprocessing, the data are used by the AI layer to estimate filter condition and remaining life and to derive an operational risk level.

The user-facing layer then communicates the results. Maintenance personnel can review current readings, historical trends, filter health, risk status, alert history and sensor status, and public users can access tap-specific information through QR codes.

Design Principles

- Measurements should be traceable to a time and location.
- Raw sensor values should be cleaned and transformed before model use.
- AI outputs should be interpreted as operational decision support.
- Alerts should be visible locally and through the monitoring interface.
- **Modularity:** the sensing layer can be understood independently of the AI layer, and the dashboard can consume outputs without knowing how the models operate internally.
- **Responsible interpretation:** prototype indicators are distinguished from laboratory safety assessment.

Comparison of Existing and Proposed System

The comparison between existing monitoring and PUREXAI is most useful when the criteria are explicit. Basic systems may provide periodic samples or live sensor readings, while the proposed workflow combines regular acquisition with tap-level history, filter-health estimation, remaining-life prediction, operational risk scoring, alerts and QR-based information. The comparison should not imply that every existing system lacks these functions; rather, it identifies the capabilities that the PUREXAI prototype brings together within the documented design.

Continuous sampling can improve temporal coverage compared with occasional inspection, but it does not remove the need for data-quality checks. More frequent readings can also create more records that need to be stored and interpreted. The proposed system addresses this by defining a one-minute interval and by adding preprocessing and derived indicators. The suitability of that interval still depends on the physical system and should be reviewed if the prototype is adapted to a different facility.

Predictive maintenance is another point of difference. A raw measurement dashboard shows what the sensors report, while FHI and remaining-life outputs attempt to summarize filter condition and future operating time. These estimates can help organize inspection, but they

require validation against independent maintenance evidence. The report therefore treats them as decision-support outputs rather than guaranteed diagnoses or replacement dates.

The proposed framework also combines local and remote communication. The OLED and buzzer can present a warning near the tap, while the dashboard and QR page provide other forms of access. This multi-channel approach can improve visibility, but each channel must preserve the correct tap identity and data freshness. A future controlled comparison could evaluate whether this combination reduces investigation time or missed events relative to a simpler threshold-only system.

Table 2 compares the characteristics of conventional monitoring with the proposed framework.

Table 2 : Comparison of Existing and Proposed System

Aspect	Existing monitoring	PUREXAI
Data collection	Periodic or sensor-only	Continuous, 60-second sampling
Data association	Often unspecific	Timestamp and tap/facility identifier
Filter assessment	Operator interpretation	FHI with Healthy/Warning/Critical bands
Replacement planning	Fixed schedule	Remaining-life estimate (Random Forest regression)
Prioritization	Manual	WQRS with Low/Moderate/High/Critical bands
Alerts	Dashboard only or absent	OLED, buzzer and platform alerts
Public information	Limited	Tap-specific QR access

Advantages

- Continuous, tap-level record for trend analysis.
- Maintenance-oriented outputs instead of raw readings alone.
- Local and platform alerts with measured mean latency of 2.1 seconds.
- Low-cost, compact embedded hardware.

Applications

The proposed system applies to public drinking-water taps and similar facility locations for maintenance planning, inspection prioritization, public information and operational record keeping.

3.3 System Architecture

A layered architecture reduces coupling between the sensing device and the user interface. The embedded controller does not need to know how the dashboard draws a graph, and the dashboard does not need to know how the analog sensor converts a signal. Instead, each layer communicates through defined records and outputs. This separation makes it easier to test components and replace a module, provided that the data contract remains stable and the meaning of fields is preserved.

The data path should be observable from end to end. Each observation should carry enough information to identify its source and acquisition time, and downstream outputs should retain a link to the observations used to produce them. This traceability is especially useful when a risk alert is reviewed later. A technician can determine which measurements were available, whether the record was delayed and which operational indicators were calculated from it.

The system should define how it responds when a layer is unavailable. If Wi-Fi is disconnected, the embedded node may be unable to deliver a new record; if the storage service is unavailable, the dashboard may lack current data; and if model inference fails, the system may still have raw readings but no new prediction. These conditions should be represented explicitly. A degraded state is more informative than a fabricated normal output and allows a user to decide whether manual inspection is required.

The architecture can be extended to more nodes by assigning each installation a stable identifier and validating incoming records against it. Additional nodes increase the number of concurrent observations and the need for clear dashboard filtering. The current experiment uses three nodes, so scaling behavior has not been established by the reported results. A future multi-site evaluation should test simultaneous reporting, data isolation, alert routing and the ability to maintain clear records across facilities.

The architecture is layered to maintain a clear path from physical measurement to decision support. At the bottom, sensors interact with the monitored water and produce signals. The ESP32 samples those signals and handles the first stage of processing. Wi-Fi and the HTTP/REST communication path transfer structured observations to the software service. A data-management component stores the records, while preprocessing prepares them for model inference. The analytical and risk components then produce the operational outputs, and the dashboard, local display, buzzer and QR interface communicate selected results.

The architecture should preserve both the raw measurement and its processed representation where storage capacity and system design permit. Raw records are useful for auditing, recalculating features and investigating whether a preprocessing rule distorted a signal.

Processed features are useful for model inference and can reduce repeated computation. If only the processed value is retained, it may be difficult to reconstruct why a model produced a particular output. A future version should document retention rules, version the preprocessing procedure and make it possible to associate a model prediction with the input record used to produce it.

The communication interface is a boundary between embedded firmware and backend services. A consistent API should specify required fields, accepted units, validation errors and response behaviour. A malformed record should be rejected or flagged rather than silently stored as valid. The service should also handle duplicate messages, since network retries may cause a record to arrive more than once. An idempotent record identifier or a combination of source identity and acquisition time can help manage duplicates. These are robust-design considerations for future hardening; the report does not claim that every such mechanism is already implemented.

The architecture can be scaled by adding more tap nodes, but scaling requires more than attaching additional sensors. Each node needs a unique identifier, an installation record, a calibration history and a method for reporting connectivity and health. The backend must handle concurrent incoming records, the dashboard must support selection by location, and alert policies must avoid confusing one tap's status with another. The current three-node evaluation is a useful prototype configuration, while larger deployments should be tested for data volume, reliability, access control and the clarity of multi-site presentation.

Architectural Overview

The system architecture contains water sensors, an ESP32 edge node, Wi-Fi communication (HTTP/REST API), time-series database records, an AI engine, a risk engine and a dashboard or user interface. Data move from the physical sensors through the edge node and communication layer before being analyzed and displayed. The AI engine performs filter-health, anomaly-oriented and trend-oriented analysis; the risk engine performs risk analysis, threshold checking and alert generation; and the dashboard provides live monitoring, graphs, reports, alerts and notifications. Figure 1 shows the architecture.

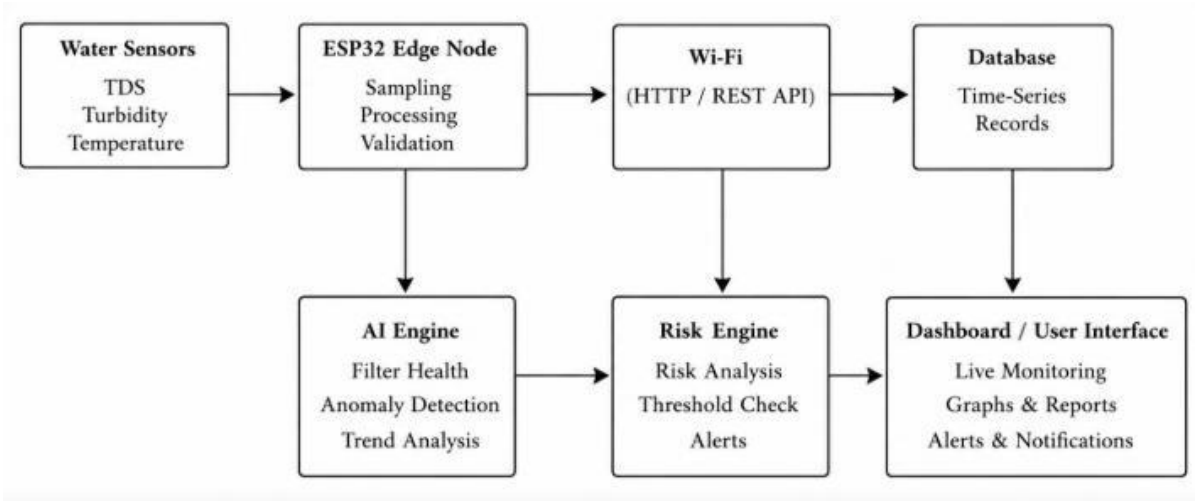


Figure 1 – PUREXAI system architecture: sensing, edge processing, communication, AI analysis, risk assessment and user interfaces

Layers of the Architecture

1. **Sensing Layer:** the TDS, turbidity and temperature sensors provide the raw observations.
2. **Edge/ESP32 Layer:** the ESP32 samples the sensors, performs basic processing and validation, and prepares observations for transmission.
3. **Communication Layer:** Wi-Fi with HTTP/REST API communication transfers timestamped observations to the software system.
4. **Data Management Layer:** time-series records retain observations for current views, trends and model inputs.
5. **AI and Decision Layer:** the AI engine and risk engine produce FHI, remaining life, WQRS and alert conditions.
6. **User Interface and Alert Layer:** the dashboard, OLED, buzzer and QR-based access present results to users.

The functions of each layer are summarized in Table 3.

Table 3 : Layer-wise Functions of the PUREXAI Architecture

Layer	Components	Function
Sensing	TDS, turbidity, DS18B20 sensors	Measure water indicators
Edge/ESP32	ESP-WROOM-32	Sampling, validation, transmission
Communication	Wi-Fi, HTTP/REST API	Transfer observations
Data management	Time-series records	Store and serve observations
AI and decision	AI engine, risk engine	FHI, remaining life, WQRS, alerts
User and alert	Dashboard, OLED, buzzer, QR	Present results and warnings

Data Flow

The data flow starts at the water sensors. TDS, turbidity and temperature are acquired by the ESP32, which performs sampling and basic processing before sending observations through Wi-Fi. Stored observations are made available to the AI and risk engines. The AI engine produces filter-health and maintenance-oriented outputs, while the risk engine combines those results with current and trend information; the final outputs are passed to the dashboard and alert mechanisms. The sequence is: sensor → ESP32 → communication → storage → preprocessing → AI → risk → dashboard/alerts/QR. Because each record carries a timestamp and tap identifier, the system can reconstruct trends and associate decisions with a specific monitored unit.

3.4 Hardware Requirements

The ESP-WROOM-32 controller is the central acquisition and communication component. The TDS sensor and turbidity sensor provide analog measurement signals, while the DS18B20 provides a digital temperature measurement. The OLED display and buzzer provide local output, and the supporting wiring, resistors, breadboard and power supply connect the components into a working prototype. Each component has a distinct role, so the hardware design should be documented in terms of the signal it provides, the interface used, the power requirements and the checks needed to determine whether it is functioning.

Sensor interfaces must be compatible with the controller's electrical limits. Analog sensor outputs need to be connected to appropriate input pins and interpreted using the required conversion procedure. The DS18B20 uses a digital communication interface and requires correct wiring and the supporting components recommended for its configuration. The report identifies the components but does not provide a complete pin-by-pin wiring diagram or measured electrical characterization. Those details should be documented from the actual prototype rather than inferred, because incorrect pin assignments or voltage assumptions can damage hardware or produce invalid readings.

The local display should prioritize information that can be understood quickly. Useful fields include the latest measured values, a concise status, and an indication of sensor or communication failure. The buzzer should be driven through an appropriate interface and should not be activated so frequently that users ignore it. Alert behavior should distinguish a short transient from a sustained condition where the configured logic supports that distinction.

The exact acoustic pattern, display layout and debounce policy should be documented if they are part of the implemented firmware.

A breadboard is convenient for assembly and experimentation, but it is not necessarily suitable for long-term installation in a public facility. Loose connections, moisture exposure and accidental contact can affect readings or interrupt operation. A field-oriented design would use a protected enclosure, secure terminals, labeled connectors, strain relief and an appropriate power arrangement. It should also allow the sensors to be removed for cleaning or calibration without requiring the entire system to be dismantled. These considerations form part of engineering readiness even though the current report focuses on a prototype.

The hardware design uses a compact embedded controller and a set of sensors selected for the prototype. The controller is an ESP-WROOM-32. The monitored parameters are obtained through a DFRobot Gravity Analog TDS Sensor, a DFRobot Gravity Analog Turbidity Sensor and a DFRobot Gravity Waterproof DS18B20 Temperature Sensor Kit. The prototype also includes a 0.96-inch 128×64 OLED display, a buzzer driver, LEDs, resistors, a solderless breadboard, jumper wires, a power supply and the public drinking-water or filter assembly. Table 4 lists the principal components.

Table 4 : Hardware Components Used in the Prototype

Component	Model / Technology	Purpose
Controller	ESP-WROOM-32	Sensor acquisition and communication
TDS sensor	DFRobot Gravity Analog TDS Sensor	TDS measurement
Turbidity sensor	DFRobot Gravity Analog Turbidity Sensor	Turbidity measurement
Temperature sensor	DFRobot Gravity Waterproof DS18B20 Temperature Sensor Kit	Water-temperature measurement
Display / alert	0.96-inch 128×64 OLED and buzzer driver	Local status and warning indication
Supporting items	LEDs, resistors, solderless breadboard, jumper wires, power supply	Wiring, indication and power

The hardware arrangement supports sensor acquisition, local status indication and wireless communication while keeping the prototype compact enough for practical experimentation.

3.5 Software Requirements

The software requirements follow the functions visible in the implemented framework: an embedded acquisition program for the ESP32, a communication interface, storage for

timestamped time-series records, machine-learning components for classification and regression, a monitoring dashboard, an alerting mechanism and a QR-based interface for tap-specific information. The dashboard screens show live monitoring, historical information, water-quality status, sensor health, alert history, AI analysis, filter health, maintenance and reports as user-facing functions.

The project documentation does not provide a complete software-version inventory. Accordingly, Table 5 describes the software requirements by function rather than listing package versions or dependencies that are not supported by the source material.

Table 5 : Software and System Requirements

Software / Function	Role in PUREXAI
Embedded acquisition	Reads TDS, turbidity and temperature through the ESP32
Wireless communication	Transfers timestamped observations through Wi-Fi (HTTP/REST API)
Time-series storage	Retains observations for historical analysis
Machine-learning engine	Supports filter-state classification and remaining-life estimation
Risk engine	Calculates the operational Water Quality Risk Score and alert conditions
Dashboard	Displays readings, trends, status, alerts and reports
QR interface	Provides tap-specific public information

CHAPTER 4 – IMPLEMENTATION

The implementation converts the layered design into a working prototype. The physical prototype is built around an ESP32 controller and three water-quality sensors, while the software side provides data transfer, storage, machine-learning analysis, risk scoring, alerts and dashboards. The implementation is described as a connected sequence: sensor readings become timestamped observations, the observations are cleaned and transformed into features, the AI modules generate maintenance-oriented indicators, and the user interfaces communicate the resulting status. The prototype was evaluated over three public-tap nodes for 10 days with a one-minute sampling interval, producing 43,200 observations.

4.1 Methodology

The methodology proceeds from requirement analysis to hardware integration, acquisition, communication, preprocessing, model training, risk calculation, alerting, interface development and evaluation. Each stage should produce evidence that can be checked before the next stage is assessed. For example, the acquisition stage should first be verified for correct sensor reads, while the analytical stage should be evaluated only after the data and target definitions are documented. This staged approach helps isolate faults and makes the results easier to reproduce.

The experimental configuration uses three public-tap nodes and a 60-second sampling interval over ten days. This produces 43,200 timestamped observations under the stated sampling plan. A chronological 70/15/15 split assigns 30,240 observations to training, 6,480 to validation and 6,480 to held-out testing. The split is appropriate for examining later observations in a time-ordered record, but the experiment should also document feature windows and target construction to prevent leakage through preprocessing or label definition.

The evaluation includes classification metrics, a regression error measure and an alert timing measure. These metrics are complementary rather than interchangeable. Classification precision, recall and F1-score summarize agreement with categorical labels; MAE summarizes the average absolute difference between a numerical prediction and its target; and latency measures response time. Each result needs its own definition and test context so that a reader can understand what was measured and what remains untested.

The methodology should also distinguish experimental facts from recommended practice. The dataset size, sampling interval and reported metrics are project results. Independent laboratory validation, longer-term deployment, more sensors, detailed failure-injection tests and controlled baseline comparisons are future work unless separate records show that they have

been completed. Keeping this distinction clear prevents the report from presenting proposed enhancements as completed experiments.

Requirement Analysis and System Design

Requirement analysis translates the operational problem into testable system behavior. A functional requirement states what the system should do, such as acquire the three measurements, store the record, calculate the risk band or display an alert. A non-functional requirement describes a quality such as reliability, usability, maintainability, security or response time. Separating the two helps prevent a system from being judged only by whether a feature exists. For example, an alert function may exist, but its usefulness also depends on latency, clarity and the ability to associate it with the correct tap.

The prototype's primary functional requirements are regular sensing, timestamped data transmission, historical record retention, model inference, risk calculation, alerting, dashboard monitoring and QR-based information access. The main non-functional considerations include data integrity, clear status presentation, reasonable response time, modularity and maintainability. The reported mean alert latency of 2.1 seconds across 100 events provides one measured performance indicator. Other quality dimensions, such as long-term uptime, ease of installation and user comprehension, require separate testing and should not be inferred from the latency result.

A requirements traceability approach can connect each requirement to its implementation and evaluation. For example, the requirement to preserve tap identity can be traced to the record schema and tested by checking whether readings from different nodes remain distinguishable. The requirement to produce timely warnings can be traced to alert-generation logic and measured by recording the time between the trigger and the displayed or delivered alert. The requirement to show sensor health can be traced to the health interface and tested by introducing a controlled sensor or communication failure. Such traceability makes the report more useful as an engineering document and helps future developers identify what has and has not been verified.

The system design should also identify assumptions. The prototype assumes that the sensors are connected and can provide usable measurements, that timestamps and tap identifiers are available, and that the configured model and risk logic are accessible to the monitoring software. If any assumption fails, the user interface should make the failure visible. Documenting assumptions prevents a design description from being mistaken for a guarantee of uninterrupted operation under every environmental or network condition.

The requirements were derived from the operational problems identified in Chapter 1: continuous observation, tap-level traceability, filter-condition assessment, remaining-life estimation, risk prioritization and timely alerts. These requirements were translated into the six-layer architecture of Chapter 3.

Sensor Integration and Data Acquisition

The acquisition stage is the first point at which the physical environment becomes a digital record. For that reason, it should be designed to preserve both the measurement and the conditions under which it was obtained. A reading should be associated with its parameter name, expected unit, acquisition time and source node. If the sensor library returns an error or the value cannot be parsed, the firmware should retain a fault indication rather than treating an arbitrary number as a valid measurement.

The sensors used in PUREXAI have different interfaces and measurement characteristics. The TDS and turbidity sensors provide analog outputs that require conversion, while the DS18B20 temperature sensor communicates digitally. This means that a shared sampling cycle still needs parameter-specific acquisition logic. The system should apply appropriate validation to each channel and should avoid assuming that the same numeric range or error behavior applies to every sensor.

It is useful to separate a valid zero, a missing measurement and a failed read. These conditions have different meanings for downstream analysis. If all three are represented by the same value, the preprocessing stage may interpret a sensor fault as a real observation. A quality flag or explicit missing value helps preserve the distinction. The dashboard can then display the latest valid measurement while also informing the user that another channel needs attention.

The sampling interval should be documented alongside the acquisition sequence. In this prototype, measurements are collected every 60 seconds. This provides regular observations for historical trends and creates the reported 43,200-observation dataset across three nodes and ten days. A future implementation should also specify the handling of missed cycles, restarts and delayed transmissions so that the stored time series reflects the actual acquisition history.

The methodology begins with continuous measurement of TDS, turbidity and water temperature. The ESP32 acquires the signals and associates each observation with a timestamp and tap or facility identifier, creating a structured record that can be grouped by tap and analyzed over time. The prototype uses the three sensors summarized in Table 6.

Table 6 : Sensor Configuration Used in Prototype

Parameter	Sensor / Model	Range	Sampling	Calibration / Reference
TDS	DFRobot Gravity Analog TDS Sensor	0–1000 ppm	60 s	Reference solution
Turbidity	DFRobot Gravity Analog Turbidity Sensor	0–1000 NTU	60 s	Reference standard
Temperature	DFRobot Gravity Waterproof DS18B20 Temperature Sensor Kit	−10 to 85 °C	60 s	Reference thermometer

Reference solutions, a reference standard and a reference thermometer are identified as calibration or reference materials. The supplied experimental record does not provide a separate numerical calibration-error analysis.

Data Transmission and Storage

The communication stage transfers records from the embedded device to the monitoring software. A useful message format is predictable and versioned, so that changes to field names or units do not silently break the receiving service. The server should validate the source identifier, timestamp and measurement fields before storing a record. Invalid messages should generate diagnostic information, while the user-facing interface should avoid exposing internal stack traces or other implementation details.

Network delivery is not always immediate. Wi-Fi interruptions, service restarts or repeated requests can result in delayed or duplicate records. A robust design should define whether the device retries a failed transmission, whether it buffers observations locally and how duplicate messages are recognized. If buffering is not supported, the interface should make data gaps visible. These behaviors should be verified from the actual firmware and backend before being described as implemented features.

Time-series storage is valuable because it allows a current reading to be compared with earlier observations. Queries by tap and time range can support trend plots, alert investigations and maintenance review. A record should retain the original acquisition timestamp even if it reaches the server later. Where practical, the system can also retain a server-receipt time, making it easier to measure transmission delay separately from model and alert processing time.

Data retention should match the expected monitoring period and the purpose of the stored history. Longer operation increases storage needs, but indiscriminate deletion can remove the context needed to review a maintenance event. A future deployment should define backup, retention and recovery procedures and should document how data are exported for model

evaluation. These procedures are especially important if the historical record is used to support operational decisions or future model training.

A time-series record typically contains a source identifier, a timestamp and a set of measurements. For PUREXAI, the central measurement fields are TDS, turbidity and temperature. Additional metadata may include a quality flag, sensor status and communication information where available. The exact field names and database schema should be taken from the implementation; this report describes the logical structure rather than inventing a schema that may not match the source code. A stable schema is important because the dashboard, feature-generation code and model inference pipeline all depend on consistent field meanings.

Time-series storage supports several operational tasks. It allows the latest reading to be retrieved for the dashboard, historical observations to be plotted, the context around an alert to be reviewed and the model inputs to be reconstructed. Queries should use the tap identifier and time range so that records from different locations are not mixed. The system should also define how duplicate, late or out-of-order observations are handled. A record received after a network interruption should retain its acquisition time so the trend remains chronologically meaningful.

Data retention and backup should be considered as part of system operation. A small prototype may store a manageable amount of data locally or in a lightweight service, but a longer deployment can accumulate a substantial history. The expected data volume depends on the sampling interval, number of nodes, fields retained and retention duration. A future implementation should specify retention periods, backup frequency and recovery procedures, particularly if historical records are used to support maintenance decisions. These policies should be based on the actual deployment and should not be described as already implemented unless the repository or operating procedure confirms them.

Data integrity also matters during analysis. The system should preserve original observations and document preprocessing steps so that a result can be reproduced. If an observation is removed because it is invalid, the reason should be recorded where practical. If a moving median is applied to reduce short spikes, the raw and filtered values should not be confused. A clear separation between source data, cleaned data and derived features makes it easier to investigate unexpected model outputs and supports a more transparent evaluation process.

Observations are transferred from the ESP32 to the monitoring software over Wi-Fi using HTTP/REST API communication and stored as time-series records. The communication layer must preserve the association between an observation and its source, so timestamps and tap or facility identifiers accompany every record.

Data Preprocessing and Feature Engineering

The preprocessing stage should preserve a clear distinction between the raw measurement, the cleaned measurement and the derived feature. The raw value represents what the acquisition layer recorded. The cleaned value is the result of applying documented quality rules, while a feature may summarize recent values or express a change relative to a baseline. If these representations are mixed without labels, it becomes difficult to audit a prediction or explain why the dashboard shows a value different from the original sensor reading.

Missing-data handling should be based on the cause and extent of the missingness. A single missing reading caused by a short communication interruption may be treated differently from a sensor that has failed for an extended period. Any imputation method should be documented and should not create a false impression that a measurement was directly observed. Where no reliable imputation is possible, the system should preserve the gap and make it visible in trend views.

The moving-median procedure is intended to reduce the influence of short isolated spikes. The effect of the filter depends on the window length and whether the window is centered or trailing. A centered window can use later observations when processing a historical point, which may be inappropriate for a real-time prediction unless handled carefully. A trailing window uses current and previous observations and better reflects information available at the time of prediction. The exact implementation should be verified against the project code before the report specifies one of these behaviors as implemented.

Feature scaling should be applied consistently between training and deployment. If a scaler is fitted using the entire dataset, information from the held-out period can influence the representation of the training examples. A leakage-controlled pipeline fits scaling parameters on training data and applies them unchanged to validation and test data. The same stored parameters are then used for future observations. This process improves reproducibility and makes it easier to compare the behavior of the trained model with its behavior during later operation.

Preprocessing prepares observations for analysis while preserving the meaning of the source data. Invalid-value detection checks whether a reading is missing, unparsable or outside the configured sensor range. Missing-data handling determines whether a record can be retained with a missing field, excluded from a particular model input or processed using a justified imputation method. The appropriate choice depends on the reason for the missing value and the intended model. Treating every missing value as zero is generally unsafe because zero may represent a real measurement or a value with a distinct operational meaning.

The five-sample moving median described in the project is intended to reduce the influence of short isolated spikes. A median is less affected by one extreme observation than a simple mean, which makes it useful for impulsive disturbances. However, any smoothing procedure can also reduce or delay a real rapid change. The choice of a five-sample window should therefore be documented and assessed against the expected response time of the monitoring task. If the data are sampled every 60 seconds, a five-sample window spans five consecutive sample positions, and the precise implementation should state whether it is centered or trailing and how edge samples are handled.

Calibration links the sensor's electrical or digital output to an interpretable measurement. The report identifies a reference solution for TDS, a reference standard for turbidity and a reference thermometer for temperature. Calibration should be repeated according to a defined procedure and recorded with date, reference material and any adjustment applied. Since the current documentation does not report a separate numerical calibration-error analysis, this report does not assign an accuracy percentage to the sensors. A future evaluation should quantify calibration error over the relevant range and identify how often recalibration is needed under the expected operating conditions.

Feature engineering can add temporal context to current measurements. A moving average summarizes recent level, a measure of variability describes short-term fluctuation, and a rate-of-change feature indicates whether a parameter is rising or falling. Deviation from a baseline can show whether the current reading differs from a reference period. Filter age and accumulated operating time may provide maintenance context if these records are available and reliable. Every derived feature should have a documented definition, unit, window length and missing-data rule so that training and deployment calculate it in the same way.

Normalization places numerical features on a common scale for analytical processing. The project describes scaling selected inputs to the interval from zero to one. In a reproducible machine-learning pipeline, the scaling parameters should be estimated using the training portion and then reused for validation, testing and future observations. Calculating the minimum and maximum from the complete dataset can leak information from the test period into the model workflow. The report specifies a chronological split; future reproducibility documentation should make the fit-and-transform order explicit and verify that no evaluation-period information influences preprocessing parameters.

Before model training and evaluation, the data pass through invalid-value detection, missing-data handling, noise filtering where required, calibration, timestamp synchronization, normalization and the generation of trend and rate-of-change features (Section 4.2.2).

AI Model Training and Evaluation

The reported experiment uses three public-tap nodes and a sampling interval of 60 seconds. Over 10 days this produces 43,200 timestamped observations, which are divided **chronologically** into 70% training, 15% validation and 15% held-out test observations. Table 7 summarizes the dataset. Keeping the sequence order prevents later observations from being included in model training. Random Forest classification is used for filter state, Random Forest regression for remaining life, and evaluation uses accuracy, weighted precision, recall and F1-score, MAE, risk-classifier F1-score and alert latency.

Table 7 : Experimental Dataset Configuration and Chronological Split

Item	Value
Public-tap nodes	3
Sampling interval	60 seconds (one minute)
Evaluation duration	10 days
Total timestamped observations	43,200
Training (70%)	30,240 observations
Validation (15%)	6,480 observations
Held-out test (15%)	6,480 observations

Risk Scoring, Alert Generation, Dashboard and QR Integration

The WQRS is computed from normalized sensor deviation, inverse FHI and short-term deterioration, and mapped to operational bands. When a critical condition is identified, the prototype activates a local buzzer and OLED warning, and platform users receive the corresponding alert. The dashboard presents readings, trends, status and sensor health, and each public tap is associated with a QR code for tap-specific information.

Testing

Testing should examine the complete workflow as well as individual components. Sensor-level checks confirm that readings can be acquired; communication checks confirm that records reach the monitoring service; storage checks confirm that records can be retrieved; and interface checks confirm that the correct values and timestamps are displayed. Model evaluation is a separate test activity because it examines predictions against defined targets. A single successful dashboard screenshot cannot demonstrate all these properties, just as a high model score cannot prove that the physical sensing and alert pathways are reliable.

The documented evaluation operated three tap nodes at a 60-second sampling interval for ten days, giving 43,200 timestamped observations. The chronological split preserves the order of

observations across training, validation and testing. This arrangement is relevant for a time-series task because future observations should not be used to train a model that is evaluated on earlier points. However, the split should be interpreted together with feature-generation windows and label construction to ensure that information from the held-out period does not indirectly enter the training process.

Alert latency is measured across 100 alert events, with a reported mean of 2.1 seconds. For a stronger latency study, each event should be recorded with a trigger timestamp and a notification timestamp, and the calculation method should be stated. It would also be useful to report the range or distribution of the measured delays and the number of failed or retried events, if those records exist. These details would show whether the mean is representative or whether a small number of slow events are hidden by otherwise fast responses.

Future testing should include controlled failure scenarios. Disconnecting a sensor can test whether the device health interface recognizes missing data; interrupting Wi-Fi can test stale-data messaging and recovery; and submitting an invalid record can test input validation. Such tests should be performed safely and documented with expected and observed outcomes. The current report does not claim a complete failure-injection campaign, so these scenarios are presented as recommended validation rather than completed experimental results.

Testing consisted of operating the three-node prototype for 10 days, evaluating the models on the held-out test partition, measuring alert latency across 100 alert events, and inspecting the monitoring and sensor-health interfaces during operation. The results are reported in Section 5.4.

4.2 Implementation

The implementation is organized into seven modules corresponding to the stages of the workflow: data acquisition, preprocessing and feature engineering, the Filter Health Index, filter replacement prediction, the risk score, alerts and QR access, and the dashboard.

4.2.1 Water Quality Data Acquisition Module

TDS Sensor

Total dissolved solids is one of the three monitored indicators. The DFRobot Gravity Analog TDS Sensor supplies the TDS measurement to the ESP32 sensing node, and the recorded value becomes part of the observation vector used by the monitoring and AI layers. The project treats TDS as an operational indicator rather than a complete statement of drinking-water safety; changes in the TDS sequence can contribute to the normalized sensor-deviation component of the WQRS.

Turbidity Sensor

Turbidity is the second monitored indicator. The prototype uses a DFRobot Gravity Analog Turbidity Sensor with a stated range of 0–1000 NTU. Turbidity provides a complementary signal to TDS: a change in turbidity can alter the operational interpretation of a tap even when TDS remains comparatively stable. The framework therefore retains the indicators as separate features, and short-term changes in the turbidity series contribute to the deterioration-trend component of the risk calculation.

DS18B20 Temperature Sensor

Water temperature is the third monitored indicator. The prototype uses the DFRobot Gravity Waterproof DS18B20 Temperature Sensor Kit with a stated range of −10 to 85 °C. Temperature provides contextual information for the monitored water condition and is retained as a separate variable so that the analytical stage can consider the three indicators together.

ESP32, Sampling Interval and Data Transmission

The ESP32 connects the three sensors, manages the sampling process and provides Wi-Fi communication. All three sensors are sampled every 60 seconds. The ESP32 acquires the measurements, associates them with the appropriate observation record and transmits them to the monitoring platform.

Timestamps and Tap Identifiers

Each observation is associated with a timestamp and a tap or facility identifier. This allows the system to construct historical trends and prevents observations from different locations from being treated as one anonymous stream. Timestamped records are important for the predictive-maintenance functions because filter condition is related to operating history, and tap-level identification also supports the QR interface.

Monitoring Interface

The water-quality interface presents the latest TDS, turbidity and temperature readings together with an overall status. When the configured operating conditions indicate a problem, an UNSAFE warning is shown to make the issue visible to the technician; this status is an operational indicator within the prototype workflow and not a laboratory or regulatory determination. The interface also includes a TDS graph and updates regularly without manual browser refresh. Figure 2 shows the interface during prototype operation.

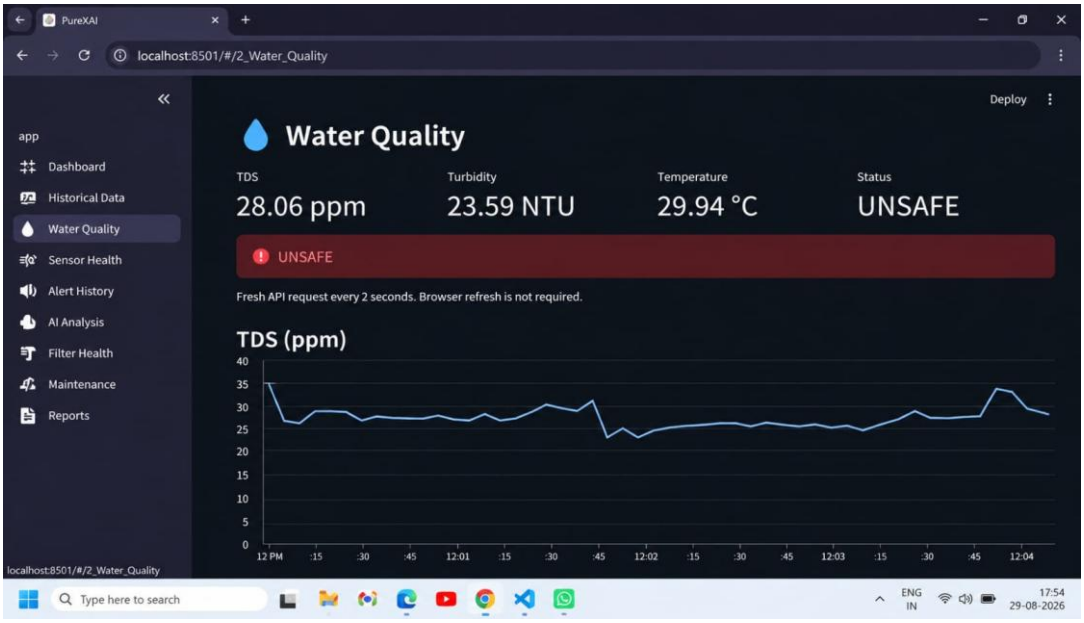


Figure 2 – PUREXAI water-quality monitoring interface during prototype operation

4.2.2 Data Preprocessing and Feature Engineering

Overview

Sensor data may contain missing values, invalid observations, measurement noise, sudden spikes or sensor-specific errors. The preprocessing stage prepares a consistent input representation for the AI models; it is not intended to create an artificial improvement in the data. The workflow includes invalid-value detection, missing-data handling, noise filtering where required, sensor calibration, timestamp synchronization, normalization or scaling, and trend and rate-of-change feature generation.

Invalid and Missing Data

Invalid-value detection prevents clearly unusable observations from entering the analytical stage. Missing-data handling is required because a continuous monitoring system can experience communication interruptions, sensor faults or incomplete records. Missing observations were handled according to the recorded data quality; the source does not specify a single universal imputation rule for every missing observation.

Noise Filtering and Moving Median

Short impulsive disturbances can appear as spikes in sensor measurements. The implementation reduces such disturbances where required using a **five-sample moving median**, which limits the influence of isolated extreme values while retaining the general shape of a short time series. It is one step within a larger sequence and is not claimed to eliminate all sensor noise.

Calibration and Timestamp Synchronization

A reference solution is identified for the TDS sensor, a reference standard for turbidity and a reference thermometer for temperature. Timestamp synchronization ensures that the three measurements are interpreted as part of the same observation record. Because no separate numerical calibration-error analysis is reported, this report does not state a sensor-accuracy percentage or imply laboratory certification.

Normalization and Scaling

Numerical inputs are scaled to the [0, 1] interval so that features with different units (ppm, NTU and °C) are processed in a consistent numerical range. Scaling does not change the physical units shown on the dashboard; it changes only the numerical representation supplied to the analytical stage. The WQRS is separately described on a 0–100 scale.

Feature Generation, Rate of Change, Trends and Baselines

The feature set can include current sensor readings, moving averages, short-term variability, rate of change, deviation from a baseline, time since the last filter replacement and accumulated filter operating time. A current TDS measurement describes the present observation, while a moving average and rate of change describe recent behavior, and filter age and accumulated operating time provide maintenance context. Feature engineering therefore acts as the bridge between raw sensing and AI decision support.

4.2.3 Filter Health Index Module

The Filter Health Index is a study-defined operational representation of estimated filter condition on a 0–100 scale. A higher value corresponds to a healthier estimated state, and the prototype groups values into Healthy, Warning and Critical bands. The index gives maintenance personnel a concise summary, but it should be understood as a derived indicator rather than a direct physical measurement. Its meaning depends on the input features, model or calculation procedure, target labels and thresholds used to interpret it. These elements should be documented together so that users can understand what a change in FHI signifies.

The current bands are Healthy at 75 or higher, Warning from 50 to 74, and Critical below 50. These thresholds provide a consistent interface for the prototype but are not external regulatory limits. A value near a band boundary may change category because of a small numerical difference, so the interface should avoid implying that a single-point transition necessarily represents a sudden physical change. A future deployment could consider persistence rules or

hysteresis to prevent status flicker, but such logic should be tested and clearly documented before being treated as an implemented feature.

The reported filter-state classifier achieved 97.01% accuracy, weighted precision of 0.972, weighted recall of 0.970 and weighted F1-score of 0.968 on the held-out test partition. These metrics describe agreement with the experiment's defined classes. Their interpretation depends on class distribution, label construction and feature independence. Because the report notes that FHI is present among the recorded feature fields while the operating classes are defined from FHI bands, there is a potential circularity between the input and target. This limits the conclusion that can be drawn about independent prediction of physical filter health.

A stronger validation protocol would define filter-state targets using independent evidence such as documented inspection, maintenance events or a reference assessment that is not itself included as a predictor. The training and testing partitions should also separate filters, facilities or time periods when the goal is to estimate performance on unseen operating conditions. Performance should be reported by class as well as in aggregate, and confusion matrices should be examined to understand which states are most often confused. These recommendations do not invalidate the reported prototype result; they specify the evidence needed to support a broader claim of generalization.

Purpose and Concept of FHI

The Filter Health Index is an operational numerical representation of the estimated condition of a filter. Larger values correspond to a healthier estimated state. The index is intended for maintenance support and should not be interpreted as a laboratory measurement of filter performance or drinking-water safety. Conceptually, the index is expressed (1) as follows, where f represents the selected AI or machine-learning model:

$$FHI = f(X, Trend, FilterAge, HistoricalData) \quad (1)$$

Features Used

The classifier uses the measured sensor features (Section 4.2.2) and the filter-condition labels defined for the experiment. The data are kept in chronological order so that later observations remain outside the training set during evaluation.

Random Forest Classification

Random Forest is used as the main filter-state classification model. Practical reasons for this choice are that the model can represent nonlinear relationships among sensor features, can tolerate a degree of measurement noise, and can provide feature-importance information that

assists interpretation. The model is one component of the overall system; the main contribution remains its integration with sensing, maintenance estimation, risk scoring and user interfaces.

FHI Interpretation and Operational Bands

The FHI uses a 0–100 operational scale in the prototype, grouped into the bands of Table 8. These are **study-defined operating thresholds**; they are not external regulatory limits and do not certify that water is safe or unsafe in a laboratory or legal sense. The dashboard uses the bands to present an understandable filter condition to maintenance personnel, and the same representation contributes to the WQRS.

Table 8 : Filter Health Index Operating Bands

Band	FHI range	Operational meaning
Healthy	75 or higher	Filter in healthy estimated state
Warning	50–74	Filter condition deteriorating; review
Critical	Below 50	Filter in critical estimated state

Classification Evaluation

On the held-out test data (6,480 observations), the Random Forest filter-state classifier achieved **97.01% accuracy**, with weighted precision of 0.972, weighted recall of 0.970 and a weighted F1-score of 0.968. These values describe performance for the defined experimental filter-state classes and feature configuration; they should not be interpreted as universal performance for every water source, filter type, geographic location or sensor configuration.

Limitations

The FHI is included among the recorded feature fields while the operating classes are defined from FHI bands. The filter-state result therefore requires careful interpretation and should not be treated as evidence of independent generalization without further leakage-controlled evaluation.

4.2.4 Filter Replacement Prediction Module

Remaining Useful Life represents the estimated operating time before a filter reaches a defined replacement condition. In PUREXAI, the task is formulated as regression, so the model returns a numerical estimate rather than only a category. A remaining-life prediction is useful only when its target is well defined. The target could be based on a recorded replacement event or another documented endpoint, but the report should not imply a particular target-construction method unless it is available in the experimental records. Clear target definitions are essential

because a low numerical error can still be misleading if the target does not represent the actual maintenance decision.

The project reports a mean absolute error of 2.8 days. MAE is calculated by averaging the absolute differences between the target values and the model estimates. It is expressed in the same unit as the target, which in this case is days. MAE is straightforward to interpret, but it does not show whether the model tends to overestimate or underestimate life, whether errors are concentrated near replacement, or whether performance varies by filter type. Additional evaluation could include the distribution of absolute errors, signed error, and error at operationally important horizons, provided the original predictions and targets are available.

Remaining-life estimation depends on information about operating history. Current sensor values alone may not reveal how long a filter has been in use or whether it has recently been replaced. Maintenance records, accumulated operating time and the pattern of previous measurements can provide context if recorded consistently. A model trained without reliable maintenance history may learn correlations that are specific to the dataset rather than to the physical degradation process. The report therefore treats the current MAE as a result under the documented experimental setup and identifies larger, more diverse filter histories as an important future requirement.

The output should be communicated as an estimate with appropriate caution. A dashboard can show the predicted number of days together with the latest update time, filter-health band and recent trend. It should not automatically schedule replacement solely because a predicted date has arrived, particularly before the model has been independently validated. A technician should review the estimate alongside inspection findings and the facility's maintenance procedure. Future versions could provide uncertainty ranges or confidence indicators if those are developed and validated; such values should not be fabricated from the current single MAE figure.

Remaining Useful Life Concept and Regression Formulation

Remaining Useful Life (RUL) is the number of operating days a filter is expected to continue operating. The filter replacement module is formulated as a **regression task**: instead of assigning a categorical condition, it estimates the number of operating days remaining.

Random Forest Regression and Prediction Workflow

Random Forest Regression is used for the remaining-life prediction. The workflow retrieves the prepared observation, generates the required features, applies the regression model and returns a numerical estimate of remaining operating life, which is displayed to support

maintenance planning. Operating time, sensor trends and filter-related information provide the input features.

Evaluation: Mean Absolute Error

The remaining-life model is evaluated using the mean absolute error, defined as (2):

$$MAE = (1/N)\ \Sigma\ |\ y_i - \hat{y}_i\ | \text{----------(2)}$$

where y_i is the observed target and $\hat{y}_i$ is the model estimate. The completed evaluation reports **MAE = 2.8 days**, the average absolute difference between predicted and target remaining-life values under the evaluated conditions. It should not be interpreted as a guarantee that future predictions will have the same error.

Maintenance Planning

Maintenance planning requires the system to translate observations into information that can be acted upon. The FHI provides a broad estimate of filter condition, while the remaining-life module estimates an operating duration in days. Together, they may help personnel identify filters that need review sooner than others. However, a predicted remaining life is not a guarantee of the exact date on which a filter will fail, and a healthy band does not eliminate the need for routine inspection.

A practical maintenance process should include a record of inspection and replacement events. These events can later be compared with the model predictions to assess whether the estimates correspond to actual filter behavior. Without reliable records of when filters were installed, serviced or replaced, the model may lack essential context. The current report identifies the value of maintenance-oriented analysis, while a more complete longitudinal maintenance dataset remains an important direction for future evaluation.

The risk band can be used to prioritize review, but the response should be proportionate to the category and the available evidence. A Low score may support routine monitoring, while a High or Critical score may prompt an earlier inspection under the prototype's study-defined operating scheme. The exact response should be determined by the facility's operating procedure and should not be presented as a universal regulatory requirement.

Maintenance personnel should be able to record the outcome of an alert review. If an alert is confirmed, the follow-up action can be documented; if it is caused by a sensor fault, that should be recorded separately. This feedback could help future evaluation distinguish useful alerts from false alarms and support improvements to the thresholds. It is a recommended extension, not a claim that a complete work-order system is already present.

A remaining-life estimate provides a different decision-support signal from the categorical FHI state. It supports planning of inspection or replacement, but does not constitute a guaranteed replacement interval for another filter or facility, because filter behavior varies with usage, installation, environment and maintenance history.

4.2.5 Water Quality Risk Score Module

The Water Quality Risk Score combines selected operational indicators into a single score intended to prioritize inspection. The documented formulation is WQRS = 0.50 × D_s + 0.30 × (100 − FHI) + 0.20 × T_d, where D_s represents normalized sensor deviation and T_d represents the short-term deterioration component. The formulation assigns the largest weight to sensor deviation, a smaller weight to inverse filter health and the remaining weight to deterioration trend. The weights sum to one, allowing the components to contribute proportionally when they are represented on compatible scales.

The formula is meaningful only when each component is defined consistently. D_s must be normalized so that its range is compatible with the other terms; FHI is described on a 0–100 scale, so 100 − FHI represents lower estimated health as a larger contribution; and T_d must be scaled according to its documented definition. If the component ranges differ, the intended weights will not have the intended influence. The report identifies the formula and the role of its components, but a complete implementation record should specify the exact normalization equations, clipping behavior and treatment of missing values.

The operational risk bands are Low from 0 to 24, Moderate from 25 to 49, High from 50 to 74 and Critical from 75 to 100, as defined for the prototype. The bands translate a numerical score into a small set of actions, from routine monitoring to urgent review. They are study-defined and should not be presented as externally certified risk categories. A facility adopting the framework would need to review the bands against its operating procedures and determine whether the suggested action for each band is proportionate to the consequences of missed alerts and false alarms.

A combined score simplifies prioritization but can hide the contribution of individual variables. Two taps could have the same WQRS for different reasons: one may have a larger sensor deviation, while another may have lower estimated filter health and a worsening trend. The dashboard should therefore display the component measurements and derived indicators alongside the combined score. This makes the score more interpretable and supports

troubleshooting. The weights and bands should also be reviewed after independent field validation rather than assumed to be optimal for every location or filter type.

Concept

The Water Quality Risk Score is an operational summary of the current condition of a monitored tap. It is defined as an operational monitoring and inspection-prioritization score, **not a regulatory drinking-water quality index**. The score ranges from 0 to 100; higher values represent greater operational concern.

Components and Weights

The WQRS combines three components: normalized sensor deviation (50%), the inverse of the Filter Health Index (30%) and the short-term deterioration trend (20%). The sensor-deviation term summarizes how far current measurements are from the defined operating baseline; the inverse-FHI term increases as the estimated filter condition becomes less healthy; and the deterioration-trend term represents recent movement toward a worse operating state.

Formulation

$$WQRS = 0.50 \times D_s + 0.30 \times (100 - FHI) + 0.20 \times T_d \text{----------(3)}$$

In equ(3)D_s is the normalized sensor-deviation score and T_d is the normalized short-term deterioration-trend score, both scaled between 0 and 100. The thresholds and weights are **study-defined operational values** and should be validated and potentially recalibrated before deployment in different facilities or under different operating conditions.

Risk Bands and Operational Actions

Table 9 translates the numerical score into operational actions. The operational risk classifier achieved an F1-score of 0.93 in the reported evaluation.

Table 9 : Operational Water Quality Risk Score Bands

Risk level	Score range	Operational action
Low	0–24	Routine monitoring
Moderate	25–49	Review trend
High	50–74	Prioritize inspection
Critical	75–100	Immediate corrective action

4.2.6 Alert and QR System

The alert subsystem converts a condition identified by the monitoring logic into a message that can prompt attention. A useful alert should identify the tap, state the condition, include the time

of the relevant observation and provide enough context for the recipient to decide what to inspect. The local OLED and buzzer support notification near the physical installation, while platform alerts allow a user to review the event through the monitoring software. These channels serve different circumstances and should be coordinated so that a single event is not presented as unrelated incidents across interfaces.

Alert logic should account for data freshness and sensor health. If the most recent measurement is old because the device has lost connectivity, the interface should show a communication warning rather than calculating a new risk level from stale information without qualification. Similarly, an invalid sensor reading should not automatically be treated as evidence of poor water quality. A robust workflow distinguishes a water-condition alert from a sensor-fault alert and a communication alert. The current report describes sensor-health monitoring and alerting; exact escalation, acknowledgement and retry behavior should be confirmed against the actual implementation.

The reported mean alert latency is 2.1 seconds across 100 alert events. Latency describes how quickly the tested alert workflow responded under the conditions measured. It does not by itself establish delivery reliability over a long period, the rate of missed alerts, user response time or performance under poor network conditions. A more complete evaluation would record each event's trigger time, notification time, channel, outcome and any retry. It could then report the median, spread and maximum latency alongside the mean, as well as the number of successful and failed deliveries.

QR-based access connects a physical tap to a corresponding information page. The code should resolve to a stable tap identifier and must not accidentally direct users to another node's record. The public page should state when the last valid reading was received and explain the monitored indicators in plain language. If a sensor is offline or its data are stale, that status should be explicit. QR access improves discoverability, but it does not make the measurements more accurate or turn an operational dashboard into a certified water-quality report.

Local Alerts: OLED and Buzzer

The OLED display provides local status information, and the buzzer driver supports audible warnings. When a critical condition is identified, the prototype activates the buzzer and displays a warning on the OLED, creating a direct path from measured condition to visible and audible warning at the monitored facility.

Platform and Dashboard Alerts, Alert Conditions

Users connected to the monitoring platform can also receive the corresponding alert. The dashboard exposes current sensor values, tap details, FHI, WQRS, filter status, historical

trends, alert status and the latest communication time. Alerts are driven by the operational conditions defined in the prototype workflow, in particular the Critical operating band.

Alert Latency

The measured **mean alert latency was 2.1 seconds across 100 observed alert events**. This value describes the tested prototype behavior and is an operational indicator of how quickly the warning path responded; network conditions, deployment distance, hardware configuration and software load can affect alert behavior in other environments.

Monitoring Workflow

The monitoring workflow acquires water data from the sensors, preprocesses and validates it, monitors the water-quality parameters, analyzes the water quality, assesses the status (Safe, Warning or Unsafe within the prototype workflow) and then triggers an alert, dashboard update or maintenance action. Figure 3 shows this workflow.

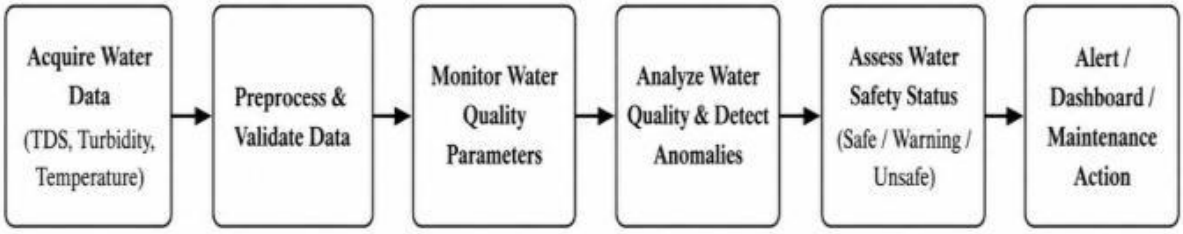


Figure 3 – PUREXAI monitoring and public-information workflow

QR Code and Public Access Workflow

The QR interface provides a convenient link between a physical tap and its corresponding information page. The code should identify the correct tap consistently, and the page should show the time of the last valid observation. If the device is offline, the interface should state that the displayed values may be stale. This is important because users scanning a code at a public facility may assume that the information reflects the current condition unless the page explicitly says otherwise.

Public-facing information should use plain language and avoid exposing unnecessary technical complexity. It can explain that TDS, turbidity and temperature are the parameters being monitored and that the system's status is an operational indicator. The page should also state that the prototype does not replace certified laboratory testing. A simple explanation can reduce the chance that a visitor interprets a risk band as an official certificate or treats the absence of an alert as proof that all water-quality hazards have been excluded.

The QR page should be designed separately from administrative functions. Public access may be appropriate for a tap identifier, recent selected readings and general explanatory text, while

configuration changes, maintenance records and device management should remain restricted to authorized users. The current report describes QR-based tap information as a system function; the exact access-control design should be confirmed against the implemented software before any security guarantee is claimed.

QR-based access can also support traceability when a user reports a concern. A stable tap identifier makes it easier to associate the report with the relevant sensor history and maintenance record. A future version could provide a feedback mechanism, but it should be designed to avoid exposing personal information unnecessarily and should not replace the formal reporting channel of the responsible facility. The value of QR access is improved discovery of tap-specific information, not independent validation of the underlying sensor measurements.

A unique QR code is associated with each public tap. Scanning it opens information related to that tap, including water-quality and maintenance details. This provides a parallel public-information path alongside the operational dashboard and separates operational monitoring from public access while keeping both views connected to the same tap-level information.

4.2.7 Dashboard and Monitoring Module

The dashboard is the main interface through which a maintenance user reviews the system's current condition. It brings together live sensor readings, status indicators, historical trends, filter-health information, risk classification, alert history and sensor status. The value of this combined view is that a user can move from a high-level summary to the underlying observations without changing systems. For example, a Critical risk band should be accompanied by the current values, the last update time and any available explanation of the components contributing to the score.

Live and historical views have different purposes. A live view emphasizes the latest observation and whether the device is currently reporting. A historical view helps users identify gradual change, repeated spikes or recurring communication gaps. These views should use consistent units and clear axes so that changes are not exaggerated by an unclear scale. Missing observations should be visible rather than interpolated in a way that suggests a continuous record where none exists. When a filtered series is shown, the interface should distinguish it from the raw measurement if both are available.

Sensor health is a separate dimension from water-quality status. A sensor-health panel can show whether readings are arriving, when the last valid record was received and whether errors have been recorded. A device may report normal-looking values while a sensor is disconnected or stuck, so a status view based only on the numeric value is insufficient. Conversely, a sensor

may be functioning correctly while a measured parameter changes. Keeping these states separate helps technicians decide whether to investigate the water system, the sensor hardware or the communication path.

The interface should support accessible and consistent status communication. Color can be used to distinguish operating bands, but the meaning should also be expressed in text so that users do not rely only on color perception. Labels such as Healthy, Warning and Critical should be applied consistently, and the same label should not mean different things on different screens. Timestamp formatting, tap identifiers and units should also remain consistent. The report documents dashboard and sensor-health interfaces; future usability work could test whether users can correctly interpret a sample alert and identify the appropriate follow-up action.

Functions and User Interaction

The dashboard is the primary user-facing interface. It presents the latest sensor readings and overall status while providing access to historical data, sensor health, alert history, AI analysis, filter health, maintenance information and reports. A user can review the current state and then move to trends, alerts or maintenance indicators when additional context is required, which reduces the need for manual interpretation of raw data. The interface updates regularly without browser refreshes.

Current Readings, Risk Status and Device Information

The live dashboard provides a consolidated view of the current system state. The captured prototype screen in Figure 4 shows TDS, turbidity, temperature, overall status, device information, the latest reading time and an updating indicator. The interface also exposes a warning state when the monitored condition is considered unsafe under the prototype's configured workflow.

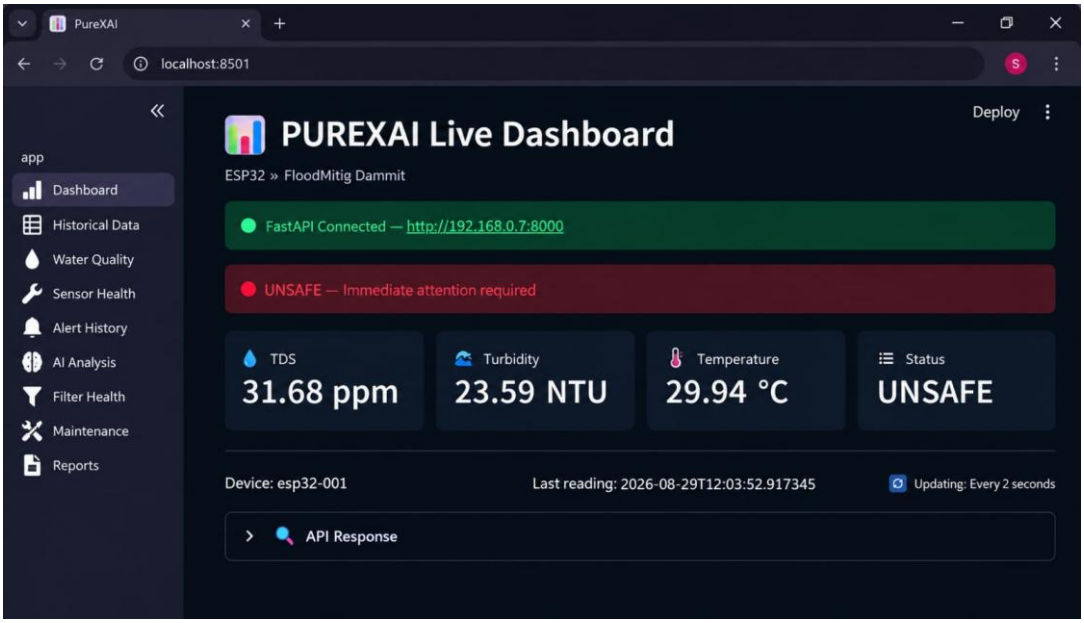


Figure 4 – PUREXAI live dashboard showing real-time sensor readings and system status

Historical Trends

Historical monitoring supports trend analysis rather than only instantaneous inspection. Timestamped observations can be displayed as sequences, allowing users to identify changes that might not be obvious from one reading. The TDS graph shown in the water-quality interface is one example; similar records can be associated with turbidity and temperature. A technician can compare the current condition with recent behavior and use the FHI, remaining-life estimate and WQRS as additional decision-support indicators.

Sensor Health and Tap Identification

The sensor-health interface (Figure 5) shows sensor status, latest readings, timestamps and error counts, allowing the user to check whether the data source itself is operating as expected. This is important because an AI decision is only as meaningful as the observations supplied to it; a missing or malfunctioning sensor can produce misleading downstream values if the problem is not identified. Tap identification allows each view to be associated with a specific monitored unit.

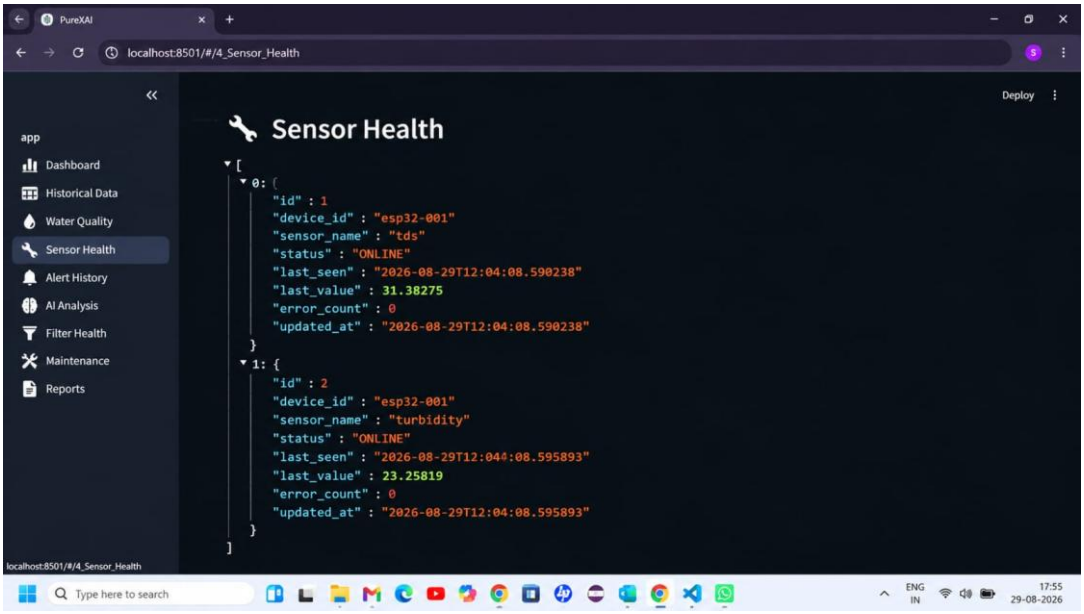


Figure 5 – PUREXAI sensor-health interface showing sensor status, latest readings, timestamps and error counts

4.3 Hardware Prototype and System Integration

Prototype Implementation

The prototype brings the hardware and software components together to demonstrate the end-to-end monitoring path. The ESP32 and sensors provide the physical measurements, the local display and buzzer communicate status near the tap, and the software side receives and retains observations for analysis. The dashboard then presents the measurements and derived outputs. The implementation should be understood as a connected system rather than a collection of separate demonstrations: a sensor reading must be traceable through transmission, storage, processing and presentation for the complete workflow to be useful.

Integration testing focuses on interfaces between components. The sensor interface must produce values that the firmware can parse. The firmware must transmit a record that the service can validate. The storage layer must retain the correct source and time. The preprocessing pipeline must receive fields in the expected units and order. The model inference stage must receive the feature representation used during training. Finally, the dashboard must display the output with the appropriate timestamp and tap identifier. Many integration faults arise when two individually functioning components make different assumptions about the same field or status.

A useful prototype test includes normal operation and controlled failure conditions. Normal operation checks whether readings arrive at the expected interval and appear in the correct interface. Failure tests can include disconnecting a sensor, interrupting Wi-Fi, sending an invalid record or verifying how stale data are displayed. The report records the 10-day

evaluation and alert-latency measurement, but it does not claim that every possible fault injection test has been completed. These additional tests are recommended for future engineering validation and should be reported as planned unless evidence exists that they were actually performed.

Physical integration also requires attention to maintainability. Sensors may need cleaning or replacement, wiring may need inspection, and the controller may require firmware updates. The installation should make it possible to identify components and reconnect them correctly after service. A maintenance log could record sensor replacement, calibration and any firmware change, because these events may affect the interpretation of historical measurements. Such operational records would strengthen the relationship between the monitoring data and the maintenance decisions that PUREXAI is intended to support.

The physical prototype integrates the ESP32 controller, water-quality sensors, breadboard circuitry, wiring and OLED display, connecting the sensing layer to the communication and local-alert functions. Figure 6 shows the hardware configuration used during prototype operation, demonstrating that the project is implemented as a physical IoT system rather than only a software model.

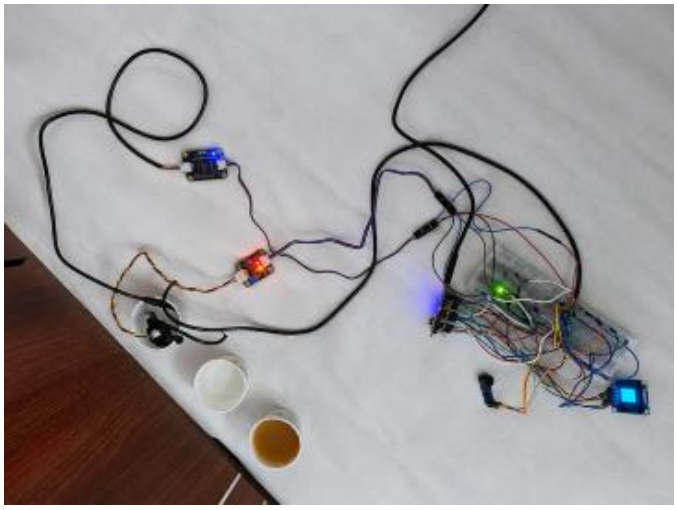


Figure 6 – PUREXAI prototype implementation with ESP32 controller, sensors, breadboard circuitry and OLED display

ESP32 Edge Node

The ESP32 is the central edge component. It connects the TDS, turbidity and temperature sensors, manages sampling and provides wireless communication so that measurements can move from the installation to the monitoring platform. The report does not add hardware specifications beyond those described in the source material.

Sensor Wiring and Local Indication

The physical implementation uses LEDs, resistors, a solderless breadboard, jumper wires and the required sensor connections. The wiring keeps the sensor outputs connected to the ESP32 acquisition layer while also supporting local feedback through the OLED and buzzer. The prototype is intentionally simple in construction, suitable for demonstrating the integrated workflow before considering a more rugged deployment design.

Communication Layer and Time-Series Records

Wireless communication transfers timestamped observations from the ESP32 to the monitoring software using Wi-Fi with HTTP/REST API communication. The system stores timestamped observations as time-series records, which allows sequences to be reconstructed, moving statistics and rates of change to be calculated and historical information to be retained for the dashboard, analytical modules and reports.

AI Engine and Risk Engine

The AI engine transforms prepared observations into maintenance-oriented information: filter-health assessment and remaining-life estimation, with additional trend and anomaly-oriented analysis represented in the architecture. The risk engine combines sensor and AI outputs to produce the WQRS, applies the study-defined risk bands and supports alert generation. The models operate on the prepared feature representation rather than directly on unprocessed raw streams, and their outputs are consumed by the dashboard rather than treated as standalone predictions.

4.4 Integrated System Operation

During a normal cycle, the sensing node acquires TDS, turbidity and temperature measurements at the configured 60-second interval. The ESP32 prepares a record with a timestamp and tap identifier and transmits it through the Wi-Fi communication path. The monitoring service validates the incoming record and makes it available to the storage and analysis components. Preprocessing handles the required quality checks and features, after which the analytical components produce filter-state and remaining-life outputs. The risk engine computes WQRS and maps it to the defined operating band, and the user interfaces present the result.

The system's output should remain linked to the input that produced it. If a technician reviews a Critical status, the software should make it possible to identify the relevant tap, observation time, sensor values and derived indicators. This traceability supports debugging and avoids the situation in which an alert is visible but its context cannot be reconstructed. For a future

implementation, a prediction record could also retain a model version and preprocessing version so that a later review can determine which logic was active when the output was generated.

The integrated workflow has several potential interruption points: sensor acquisition, device power, wireless communication, data storage, preprocessing, model inference, risk calculation and interface refresh. A robust implementation should define what happens when each stage fails. If acquisition fails, the device should report a sensor or sampling error. If transmission fails, the interface should show that the last reading may be stale. If inference fails, the dashboard should not silently reuse an old model output as a fresh prediction. If the user interface cannot refresh, the last update time should remain visible.

The prototype results provide evidence about selected aspects of the integrated workflow. The data configuration records 43,200 observations across three nodes and ten days; the model evaluation reports classification and regression metrics; and the alert test reports a mean latency of 2.1 seconds over 100 events. These are useful measurements but do not cover every reliability dimension. Longer uptime tests, failure recovery tests, independent reference measurements and a controlled comparison with simpler monitoring approaches would be needed before claiming that the framework is ready for broad deployment.

During operation, the system follows a repeating cycle. Sensors produce observations, the ESP32 acquires and transmits them, the software stores the records and the analytical layer processes the relevant features. The dashboard then presents current and historical information. At the same time, FHI, remaining life and WQRS are updated, and if the operational conditions require attention, local and platform-level alerts are generated. The integrated workflow demonstrates the central design goal of PUREXAI: continuous physical monitoring should lead to structured analytical outputs and actionable information rather than stopping at raw measurements.

CHAPTER 5 – CONCLUSION AND FUTURE WORK

The project demonstrates a practical integration of sensing, wireless communication, data handling, machine-learning analysis and user-facing monitoring. Its contribution is not the invention of a new sensor or a new machine-learning algorithm, but the connection of multiple functions in a single operational workflow. TDS, turbidity and temperature readings are collected at defined intervals, associated with tap identifiers and timestamps, and used to support a filter-health representation, remaining-life estimate, risk score and alert process. This integrated view is the central outcome of the project.

The filter-state classifier's reported accuracy of 97.01% and weighted F1-score of 0.968 indicate strong agreement with the study-defined classes on the held-out partition. The remaining-life model's MAE of 2.8 days provides a measure of the average absolute prediction error under the experiment's target definition. The operational risk classifier's F1-score of 0.93 and the mean alert latency of 2.1 seconds across 100 events add further evidence about the tested workflow. These metrics should be read together with the data configuration and limitations rather than as universal performance guarantees.

A key conclusion is that the quality of the evaluation depends on the design of the target labels and the independence of the test data. The report identifies that the FHI appears among the recorded feature fields while classes are defined from FHI bands. This potential circularity means that the high filter-state classification score cannot by itself establish independent prediction of physical filter condition. Future evaluation should separate predictors from target construction and test on independent filters, locations or periods. Transparent reporting of this limitation is necessary for an academically defensible result.

PUREXAI should be regarded as a prototype for operational decision support, not as a certified drinking-water safety system. The three monitored parameters do not provide a complete laboratory assessment, the FHI thresholds and WQRS weights are study-defined, and the evaluation covers three nodes over ten days. Within those boundaries, the project illustrates how an IoT platform can make measurements more accessible and connect them to maintenance-oriented indicators. Broader deployment should follow only after calibration, validation, reliability, security and operational procedures have been strengthened.

5.1 Conclusion

Problem Addressed and Proposed Architecture

PUREXAI addresses the difficulty of monitoring public drinking-water facilities when measurements are periodic and interpreted manually. It demonstrates an integrated approach

in which IoT sensing, machine learning, predictive maintenance, risk scoring, alerts and user interfaces are treated as one connected system.

IoT and AI Layers

The prototype measures TDS, turbidity and temperature through an ESP32-based sensing node, transfers the observations through Wi-Fi, stores timestamped records and analyzes the data using Random Forest models and an operational risk framework. The completed evaluation provides quantitative evidence for the technical feasibility of the integrated prototype under the reported three-tap, 10-day conditions.

Filter Health Index

The Random Forest classifier distinguished the study-defined Healthy, Warning and Critical classes with high measured performance (97.01% accuracy, weighted F1-score 0.968) on the held-out test observations. The FHI gives a compact operational representation of filter condition. However, because FHI is among the recorded feature fields while the operating classes are defined from FHI bands, the result should not be treated as evidence of independent generalization without further leakage-controlled evaluation.

Remaining Useful Life

The remaining-life module extends monitoring from classification to maintenance planning. The reported MAE of 2.8 days provides a numerical measure of prediction error under the evaluated conditions. The estimate remains an operational prediction rather than a guarantee, and broader validation is required before it is treated as a general replacement rule.

Water Quality Risk Score

The WQRS provides a combined operational view of sensor behavior and filter condition, mapped to Low, Moderate, High and Critical bands. The operational risk classifier achieved an F1-score of 0.93, supporting the feasibility of the defined risk-classification workflow without making the WQRS a regulatory water-quality index.

Dashboard, Alerts and QR Access

The dashboard, OLED, buzzer and QR-based tap information provide complementary routes by which maintenance personnel and the public can access the same tap-level data. The measured mean alert latency of 2.1 seconds over 100 alert events indicates that the alert path operated with low latency under the tested conditions.

Security, Reliability and Responsible AI

Even a prototype that primarily measures sensor data should consider basic security. Device identifiers, network credentials and service endpoints should not be unnecessarily exposed in public-facing pages or shared screenshots. A deployed version should define who can view operational history, change thresholds, register a device or access maintenance records. Public QR pages can show selected information without granting access to configuration controls. The current report describes a monitoring framework, but specific authentication mechanisms should only be claimed if they are confirmed by the implemented code.

Reliability requires visible failure states. When a sensor is disconnected, a network is unavailable or a model cannot produce an output, the interface should communicate the problem rather than displaying an old result without qualification. The last valid timestamp is a simple but important indicator of freshness. A clear failure state supports safer human decisions because it tells the user when the system lacks enough information to provide a current interpretation.

Responsible AI also requires that the purpose and limits of model outputs be communicated. The FHI and WQRS are operational indicators defined for this prototype, and model outputs depend on the data and assumptions used during development. Users should be able to distinguish measured values from predicted or derived values. Where model confidence or uncertainty is not calculated, the interface should not invent a confidence percentage. A clear statement that a result is an estimate is more defensible than unsupported precision.

Human oversight remains important because the system operates in a context where incorrect interpretation could have consequences. An alert should initiate inspection or appropriate follow-up rather than replace established procedures. Model updates and threshold changes should be recorded, and any new version should be evaluated before use. These measures support accountability and make it possible to explain why a decision-support output changed over time.

Because PUREXAI is a decision-support prototype for public infrastructure, reliability and responsible operation are important. The implementation considers secure communication, access control, protection of stored information, sensor-failure detection, model monitoring, periodic validation and clear presentation of uncertainty. AI-generated risk values support inspection and maintenance decisions and are not a substitute for certified laboratory testing or regulatory compliance assessment. The sensor-health interface helps users distinguish a genuine change in monitored conditions from a potential sensor or communication problem.

Contributions and Limitations

The primary contribution of PUREXAI is the integration of an ESP32 sensing node, timestamped data transfer, data preprocessing, Random Forest-based filter-state and remaining-life analysis, WQRS calculation, local alerts, dashboard monitoring and QR-based tap information. The design connects physical measurements to maintenance-oriented decision support and preserves a route for users to review the relevant tap and time. This contribution is best described as system integration within a prototype rather than the invention of a new machine-learning algorithm.

The evaluation provides a defined set of quantitative observations: 43,200 timestamped records from three tap nodes over ten days, 97.01% filter-state classification accuracy with weighted F1-score 0.968, remaining-life MAE of 2.8 days, risk-classifier F1-score 0.93 and mean alert latency of 2.1 seconds across 100 events. These values are useful evidence about the tested configuration. Their meaning depends on the experimental labels, target definitions and test procedure, so they should not be generalized beyond those conditions without further validation.

The main limitations include the short evaluation period, small number of monitored nodes, lack of a separate numerical calibration-error analysis and the fact that TDS, turbidity and temperature do not provide a complete laboratory assessment. The FHI bands and WQRS weights are study-defined, and the documented relationship between FHI features and FHI-based classes creates a potential circularity that needs leakage-controlled testing. The report also does not reproduce a complete original source codebase because that source code was not included in the supplied report materials.

These limitations define a realistic path for future work: longer and multi-location testing, independent reference measurements, improved calibration records, independent filter-condition labels, group-based or chronological validation appropriate to the target deployment, detailed alert reliability testing and reproducible software documentation. Addressing these items would strengthen the evidence for generalization and operational benefit. Until then, the system should remain a prototype decision-support tool and should not be described as a certified drinking-water safety system.

The principal contribution is system integration: continuous tap-level sensing combined with FHI, remaining-life estimation, WQRS, alerts, dashboard monitoring and QR-based information. The system is a prototype and is not a certification of drinking-water safety. The three monitored indicators are not a complete laboratory analysis, the evaluation covers three taps over 10 days, the FHI bands and WQRS weights are study-defined, no separate calibration-

error analysis is reported, and the results are not a controlled comparison with baseline methods.

5.2 Future Work

Future work should first strengthen the evidence behind the existing outputs before adding more functions. Independent filter-condition labels, reference measurements and a leakage-controlled evaluation would address the most important limitation of the filter-state classification result. A larger dataset is useful only if it contains meaningful variation and has clear labels; increasing the number of rows without improving target quality may not improve the reliability of the conclusions.

Longer operation across additional locations would test whether the framework behaves consistently under different conditions. A multi-location study should record the sensor configuration, installation date, calibration history and maintenance events for each node. Evaluation should report variation across locations rather than only a pooled average, because performance at one site may not represent performance at another. Where the deployment goal is a new facility, the test procedure should hold out sites or filters accordingly.

Operational testing should examine whether alerts lead to useful action. Relevant measures could include the proportion of alerts confirmed by inspection, false-alert frequency, missed events, investigation time and the clarity of the information available to maintenance personnel. These measures would connect technical metrics to the practical purpose of the framework. They should be collected prospectively using defined criteria rather than reconstructed from memory after a test period.

Any expansion should preserve responsible communication. New model outputs should be labeled as measured, calculated or predicted, and uncertainty should be shown only when it is actually estimated. QR pages should disclose data freshness and the limits of the monitored parameters. Before the system is used to support consequential decisions, it should undergo the relevant engineering, calibration, security and operational reviews for its intended environment.

Longer-Term, Multi-Location Validation and Laboratory References

The first priority for future work is to extend the observation period and the diversity of monitored sites. Ten days of operation can demonstrate that the workflow functions over a short prototype evaluation, but it cannot represent every seasonal, environmental or maintenance condition. A longer study should include different filters, water sources and operating patterns, with clear documentation of the conditions under which each observation was collected. The purpose is not merely to increase the record count; it is to capture

meaningful variation that tests whether the learned relationships remain useful beyond the original dataset.

Independent laboratory reference measurements would strengthen the interpretation of sensor data. The current system monitors TDS, turbidity and temperature, but those measurements do not cover every parameter relevant to drinking-water safety. A future validation protocol should collect appropriate reference samples and record the timing and handling of each sample so that the sensor observations can be matched fairly to external measurements. This would help quantify sensor error and determine which types of changes the operational system can reliably flag. The laboratory results should be treated as reference evidence rather than assumed to agree perfectly with every low-cost sensor reading.

Calibration procedures should be made repeatable and measurable. A protocol can specify the reference materials, the measurement points across the operating range, the number of repeated readings, the adjustment method and the acceptance criterion. Error statistics can then be reported separately for each sensor. Calibration records should include date, device identifier and any replacement or adjustment, since sensor behavior may change over time. The current report identifies calibration references but does not include a separate numerical calibration-error analysis; this is an explicit opportunity for future work.

A multi-location evaluation should also define the unit of generalization. Testing on future observations from the same three taps answers a different question from testing on a new tap, a new filter or a new geographic location. If the intended deployment is across previously unseen facilities, the test design should hold out facilities or filters rather than relying only on a time split within the same sites. Results should report both overall performance and variation across sites, because an average can conceal a model that performs well at one location and poorly at another.

The present evaluation covers three public-tap nodes over 10 days. Longer-term and geographically diverse testing, with multi-location deployment, would provide more varied operating conditions and a stronger basis for evaluating seasonal or location-dependent behavior. Independent laboratory reference measurements should also be collected so that the sensor indicators and AI outputs can be compared with external measurements rather than relying only on the prototype's internal records. Improved sensor calibration, with numerical calibration-error analysis, would further strengthen the data.

Leakage-Controlled Evaluation and AI Enhancements

Future model evaluation should begin with an explicit target-definition document. For filter-state classification, the labels should be based on a reference procedure that is independent of

the predictor variables. If a class is derived from the FHI, the FHI should not also be used as an input feature for a model whose purpose is to predict that same class. This separation is essential to avoid circular learning. The report's current limitation concerning FHI and class bands should guide the next experiment rather than being treated as a minor documentation issue.

The data split should reflect the intended use. A chronological split is appropriate when the goal is to predict later observations for the same monitored system, because it prevents later time points from being mixed into training. If the goal is to generalize to unseen filters or facilities, group-based separation should also be considered. Preprocessing parameters must be learned from training data only, and feature-generation windows must not accidentally include future observations when constructing a prediction for an earlier time. A reproducible pipeline should document these decisions and test them automatically where possible.

Model comparison should use a small set of justified baselines. A threshold-based rule, a simple persistence model or a straightforward regression baseline can help determine whether the more complex model provides a meaningful improvement. The comparison should use the same held-out observations and target definitions, and it should report relevant metrics rather than only the highest accuracy. A controlled baseline comparison is not claimed in the present report, so future work should frame it as a new evaluation rather than retroactively implying that such a comparison has already been completed.

Explainability can help maintenance staff understand why a prediction was produced, but explanatory tools should be evaluated for usefulness and stability. Feature importance can indicate which variables influence a Random Forest across the dataset, while local explanations can summarize the inputs associated with one prediction. Neither method automatically establishes causation. An explanation should be presented as information about the model's behavior, not proof that a sensor parameter physically caused filter degradation. Human review, independent validation and clear uncertainty communication remain necessary even when explainability features are added.

Future evaluation should include leakage-controlled model testing, which is particularly important for the current FHI formulation. Alternative models and feature configurations can be evaluated under strict chronological validation, with the objective of improved generalization rather than a higher single accuracy value. The remaining-life model could be strengthened with larger filter histories and maintenance events, and explainable-AI techniques could be explored where supported, building on the feature-importance information available from Random Forest.

System Expansion

A future hardware revision could improve physical durability and serviceability. A production-oriented enclosure should protect the controller and electrical connections from moisture, dust and accidental contact while allowing the sensors to be inspected or replaced. Cable entries should be secured, components should be labeled and the power supply should be selected for the actual operating environment. The design should also consider whether the installation can be cleaned without disturbing the sensor connections. These improvements would address deployment readiness rather than change the core sensing and analytics concept.

Additional sensors may broaden the set of conditions the system can observe, but adding a sensor should follow a defined requirement. Each new measurement needs an appropriate sensor, calibration method, sampling procedure, data field, preprocessing rule and validation plan. The system should not add parameters simply to increase the apparent sophistication of the project. New indicators are useful only when their measurement quality and interpretation are understood and when they contribute to a specific monitoring or maintenance objective.

The software could be extended to support multiple sites, maintenance logs, report exports and role-specific views. A multi-site dashboard would need a consistent location hierarchy and unique device identifiers. Maintenance logs could record inspections, cleaning, calibration and filter replacements, making it easier to compare predicted condition with real operational events. Report exports should include the time range, data completeness and relevant caveats so that screenshots or summaries are not separated from the context needed to interpret them.

Deployment architecture should be chosen according to the intended scale and connectivity. A small prototype may use a relatively simple service, while a larger deployment may need stronger monitoring, backup, authentication and capacity management. The design should avoid making the sensor node depend on a single unmonitored service without a recovery plan. If cloud hosting is considered, data access, service availability, network costs and responsibility for maintenance should be documented. These are possible enhancements; the report does not claim that cloud deployment or production-grade access control is already implemented.

Additional water-quality sensors could be added when justified by the target deployment and supported by calibration and validation. The physical prototype can be redesigned for field deployment with a production-oriented enclosure, improved wiring and protected power. The software can be extended with cloud deployment, a multi-site dashboard, richer historical reporting, maintenance records and improved alerting, while preserving the separation between measured observations, AI-generated decision support and certified external assessment.

Risk Scoring Review

The WQRS formula and operating bands should be reviewed using evidence from field operation. The present weights assign 50% to normalized sensor deviation, 30% to inverse FHI and 20% to short-term deterioration. These values express the prototype's intended weighting, but their practical usefulness depends on the behavior and normalization of the component features. A future study should examine how the score changes when each component varies, whether the score is overly sensitive to one noisy sensor and whether the resulting bands correspond to meaningful inspection decisions.

Thresholds should be evaluated in terms of operational consequences. A lower threshold may increase sensitivity but also produce more false alerts and greater inspection workload. A higher threshold may reduce alert volume while delaying attention to a developing condition. The appropriate balance depends on the costs of missed events and unnecessary investigations. Future testing should record true and false alerts against an independently defined reference, then estimate the workload associated with each threshold configuration. Such analysis would provide a stronger basis for selecting operational bands than choosing a threshold only because it creates visually distinct categories.

The score should also be tested for robustness when a component is unavailable. If FHI cannot be computed, the system should not silently substitute a neutral value and present the resulting WQRS as complete. A clearly defined degraded mode could mark the overall score unavailable while still displaying the available sensor readings and device status. If imputation is used, its rationale and effect on the score should be documented. Explicit handling of unavailable inputs prevents a numerical score from creating false confidence when the underlying evidence is incomplete.

Risk scoring should be validated separately from filter-state classification and remaining-life regression. Each output has a different target and use. The risk classifier's reported F1-score of 0.93 should be interpreted in relation to its defined classes and evaluation data, not as proof that the score predicts actual safety incidents. Future studies should define the reference outcomes for risk categories, compare the score with simple baselines and examine whether the alert bands lead to useful maintenance actions. The current score is an operational prioritization aid, and its thresholds should remain explicitly described as study-defined.

The WQRS weights and thresholds are study-defined. After field validation, future deployments should confirm whether the Low, Moderate, High and Critical bands correspond to useful operational actions at the target facilities. None of the features above has been implemented in the present prototype.

5.3 Source Code

```
#include <WiFi.h>
#include <WebServer.h>
#include <HTTPClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// Wi-Fi credentials
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// Replace with the IP address of the computer running FastAPI.
const char* API_URL = "http://192.168.1.10:8000/sensor-data";

// ESP32 ADC pins
#define TDS_PIN       34
#define TURBIDITY_PIN 35
#define ONE_WIRE_BUS  4

#define GREEN_LED  18
#define YELLOW_LED 19
#define RED_LED    21
#define BUZZER_PIN 23

WebServer server(80);
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature tempSensor(&oneWire);

float tds = 0.0;
float turbidity = 0.0;
float temperatureC = 0.0;
String waterStatus = "Starting";

void readSensors();
void evaluateWater();
void sendToBackend();
void handleRoot();
void handleApi();
void handleApi();
// Read sensors. These simple formulas are placeholders.

// Calibrate TDS and turbidity against known reference samples.
void readSensors() {
  int rawTds = analogRead(TDS_PIN);
  int rawTurbidity = analogRead(TURBIDITY_PIN);
  float voltageTds = rawTds * (3.3 / 4095.0);
  float voltageTurbidity = rawTurbidity * (3.3 / 4095.0);
  // Demonstration conversion only — not a calibrated measurement.
  tds = voltageTds * 500.0;
  turbidity = (3.3 - voltageTurbidity) * 100.0;
```

```
  tempSensor.requestTemperatures();
  temperatureC = tempSensor.getTempCByIndex(0);
  if (temperatureC == DEVICE_DISCONNECTED_C) {
    Serial.println("Temperature sensor not detected");
  }
  Serial.printf("TDS: %.1f ppm | Turbidity: %.1f | Temp: %.1f C\n",
                tds, turbidity, temperatureC);
}

void evaluateWater() {
  // Demonstration thresholds only; confirm suitable thresholds
  // for your sensors, local standards, and intended application.
  bool warning = (tds > 300.0 || turbidity > 5.0);
  bool danger = (tds > 500.0 || turbidity > 10.0);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  if (danger) {
    waterStatus = "HIGH RISK";
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
  } else if (warning) {
    waterStatus = "WARNING";
    digitalWrite(YELLOW_LED, HIGH);
  } else {
    waterStatus = "NORMAL";
    digitalWrite(GREEN_LED, HIGH);
  }
}
void handleRoot() {

  String page = R"HTML(
  <!doctype html>
  <html>
  <head>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>PUREXAI Water Monitor</title>
    <style>
      body { font-family: Arial; margin: 2rem; background: #eef6f7; }
      .card { background: white; padding: 1rem; border-radius: 12px;
              max-width: 520px; box-shadow: 0 2px 8px #bbb; }
    </style>
  </head>
  <body>
    <div class="card">
      <h2>PUREXAI Water Quality</h2>
      <p>Open <a href="/api">/api</a> to view current readings as JSON.</p>
      <p>Refresh this page to check the latest sensor status.</p>
```

```
  </div>
 </body>
 </html>
 )HTML";
 server.send(200, "text/html", page);
}

void handleApi() {
 String json = "{";
 json += "\"tds_ppm\":" + String(tds, 1) + ",";
 json += "\"turbidity\":" + String(turbidity, 1) + ",";
 json += "\"temperature_c\":" + String(temperatureC, 1) + ",";
 json += "\"status\":\"" + waterStatus + "\"";
 json += "}";

 server.send(200, "application/json", json);
}
void sendToBackend() {

 if (WiFi.status() != WL_CONNECTED) {
  Serial.println("Wi-Fi disconnected; skipping API upload");
  return;
 }

 HTTPClient http;
 http.begin(API_URL);
 http.addHeader("Content-Type", "application/json");

 String body = "{";
 body += "\"device_id\":\"PUREXAI-ESP32\",";
 body += "\"tds_ppm\":" + String(tds, 1) + ",";
 body += "\"turbidity\":" + String(turbidity, 1) + ",";
 body += "\"temperature_c\":" + String(temperatureC, 1) + ",";
 body += "\"status\":\"" + waterStatus + "\"";
 body += "}";

 int responseCode = http.POST(body);
 Serial.printf("Backend response: %d\n", responseCode);
 http.end();
}

void setup() {
 Serial.begin(115200);

 pinMode(GREEN_LED, OUTPUT);
 pinMode(YELLOW_LED, OUTPUT);
 pinMode(RED_LED, OUTPUT);
 pinMode(BUZZER_PIN, OUTPUT);

 tempSensor.begin();
 analogReadResolution(12);
```

```
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.print("ESP32 IP: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/api", handleApi);
  server.begin();
  Serial.println("Web server started");
}
unsigned long lastUpload = 0;
const unsigned long UPLOAD_INTERVAL_MS = 10000;

void loop() {
  server.handleClient();
  }

  delay(1000);
}
```

Minimal FastAPI receiver (save as main.py)

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="PUREXAI Sensor API")

class SensorData(BaseModel):
    device_id: str
    tds_ppm: float
    turbidity: float
    temperature_c: float
    status: str

@app.post("/sensor-data")
def receive_sensor_data(data: SensorData):
    print(data.model_dump())
    return {"received": True, "device_id": data.device_id}
# Install: pip install fastapi uvicorn
# Run: uvicorn main:app --host 0.0.0.0 --port 8000
```

5.4 Results

The results section should present the main measurements consistently and avoid implying that they evaluate the same task. Filter-state accuracy, precision, recall and F1-score describe agreement with the experiment's categorical labels. Remaining-life MAE describes average absolute regression error in days. The operational risk F1-score describes classification performance for the defined risk categories. Alert latency describes response time across the recorded alert events. Each metric should remain attached to its task and evaluation context.

The dataset configuration is internally consistent with the stated sampling plan: three nodes, one observation each minute, and ten days of operation produce 43,200 timestamped observations. The 70/15/15 chronological division gives 30,240 training observations, 6,480 validation observations and 6,480 held-out test observations. This arithmetic confirms the intended split counts, but it does not independently verify that every planned reading was successfully acquired or that every record passed the quality checks.

The reported 97.01% filter-state accuracy and weighted F1-score of 0.968 should be interpreted in light of the FHI label relationship documented in the report. If FHI is used as an input and the class is derived from FHI bands, the model may reproduce the band rule. A stronger result would require an independent filter-condition target and a test design that evaluates generalization to new filters or locations. The present report appropriately identifies this as a limitation and should keep the caveat adjacent to the result.

The reported MAE of 2.8 days is an average and does not describe every individual prediction. The practical value depends on the range of remaining-life targets and the amount of lead time required for maintenance. Without the underlying prediction-level data, it would be inappropriate to invent a median error, confidence interval or class-specific result. Future evaluation should report these additional measures only when the original targets and predictions are available and the calculation procedure is documented.

Consolidated Evaluation

The reported evaluation uses four principal outcomes. The filter-state classifier achieved 97.01% accuracy, weighted precision of 0.972, weighted recall of 0.970 and weighted F1-score of 0.968. The remaining-life regression model achieved MAE of 2.8 days. The operational risk classifier achieved an F1-score of 0.93. The alert workflow recorded a mean latency of 2.1 seconds across 100 alert events. These measurements describe different aspects of the system and should not be combined into one overall performance percentage because their scales and meanings differ.

The dataset configuration is three public-tap nodes, one observation per minute and ten days of monitoring, resulting in 43,200 timestamped observations. The chronological 70/15/15 split assigns 30,240 observations to training, 6,480 to validation and 6,480 to held-out testing. The time-ordered split is preferable to randomly mixing adjacent observations when the intended task concerns later readings. Nevertheless, the effective independence of the evaluation also depends on feature engineering, target generation and whether observations from the same tap or filter appear in every partition.

The filter-state metrics are numerically high, but the FHI/label relationship requires caution. If the class labels are defined from FHI bands and FHI is also a model feature, the task may partly reproduce a threshold rule rather than independently infer filter condition. A future evaluation should remove the circular feature or define an independent target, then repeat testing with the same chronological discipline. Until then, the metrics should be reported as classification performance for the experiment's defined labels, not as universal filter-diagnosis accuracy.

The remaining-life MAE should be interpreted with reference to the target distribution. An average error of 2.8 days can be more or less consequential depending on the typical life remaining and the maintenance lead time required. The report does not provide a full error distribution or confidence interval, so those should not be invented. Similarly, the risk-classifier F1-score needs its class definitions and confusion matrix for deeper interpretation. The alert-latency result is a useful response-time measure but does not quantify false alarms, missed events or long-term availability.

The most defensible conclusion is that the integrated prototype demonstrated technical feasibility under the tested conditions. The measurements show that the system collected a defined dataset, produced model outputs and delivered alerts within the measured average latency. They do not establish complete drinking-water safety, universal generalization or superiority over a controlled baseline. Future studies should strengthen external validation, publish reproducible evaluation details and assess whether the outputs lead to better maintenance decisions in real facilities.

The final evaluation is summarized in Table 10. The experimental dataset consisted of 43,200 timestamped observations collected from three public-tap nodes over 10 days at one-minute sampling.

Table 10 : Consolidated Evaluation Results of PUREXAI

Measure	Value	Task / data
Filter-state accuracy	97.01%	Held-out test data
Weighted precision	0.972	Filter-state classification

Measure	Value	Task / data
Weighted recall	0.970	Filter-state classification
Weighted F1-score	0.968	Filter-state classification
Remaining-life MAE	2.8 days	Random Forest regression
Operational risk classifier F1-score	0.93	WQRS risk classes
Mean alert latency	2.1 seconds	100 alert events

Variation of Sensor Measurements

Figure 7 illustrates the variation of TDS, turbidity and temperature measurements during prototype operation, giving a visual example of the time-series behavior used by the monitoring and analytical workflow.

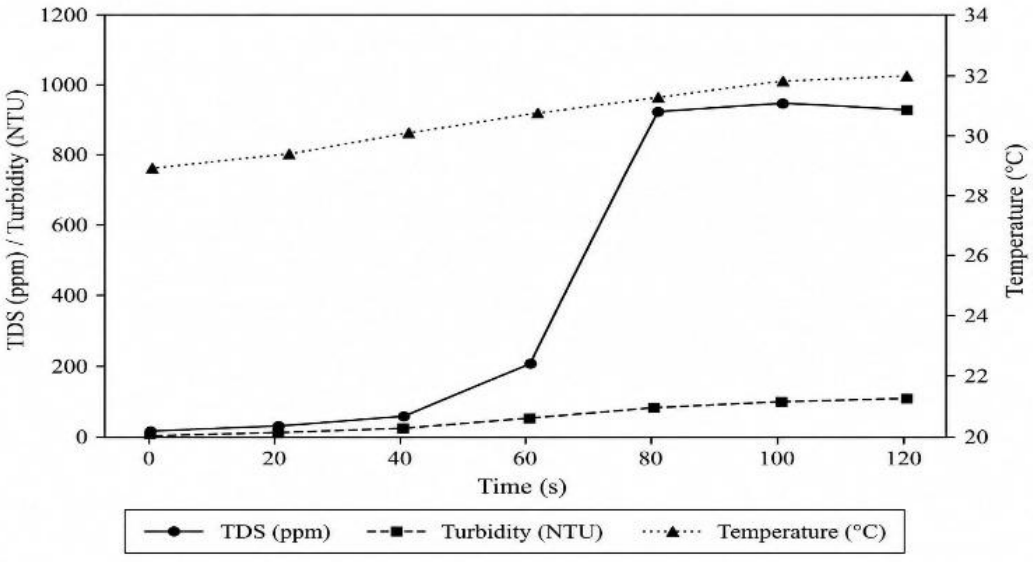


Figure 7 – Variation of TDS, turbidity and water temperature measurements obtained from the PUREXAI monitoring system during prototype operation

TDS Comparison and Interpretation

Figure 8 compares TDS values at three public drinking-water taps, labelled X, Y and Z, between selected previous studies [12], [14], [16] and PUREXAI. The figure is descriptive and provides context rather than a controlled experimental benchmark. Because the underlying datasets, locations, measurement conditions and study objectives differ, the values should not be interpreted as direct evidence that one system is universally better than another.

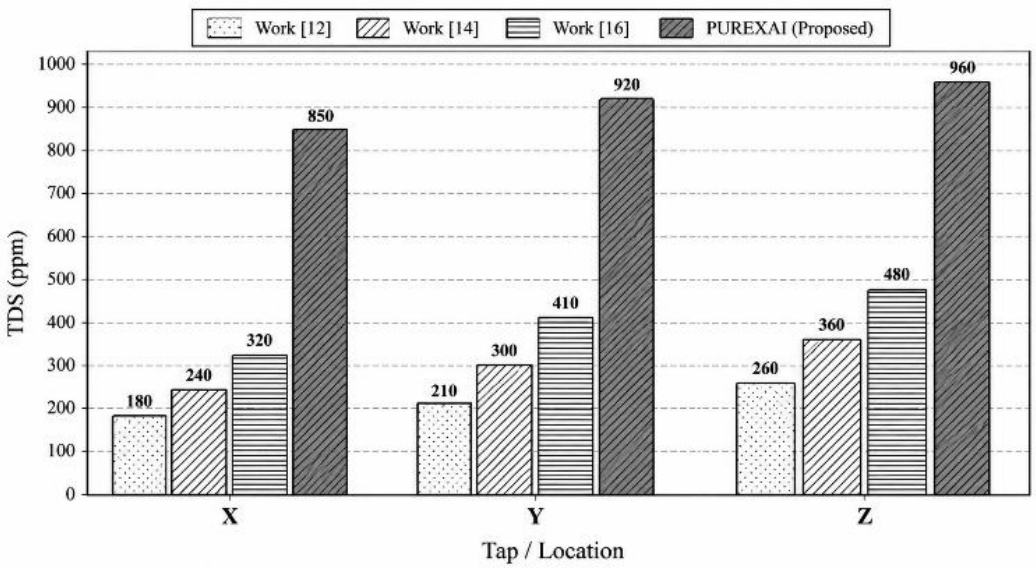


Figure 8 – Comparison of TDS values at three public drinking-water taps (X, Y and Z) between selected previous studies and the proposed PUREXAI system

Comparison of Reported Classification Accuracy

Figure 9 places the PUREXAI filter-state classification accuracy of 97.01% alongside values reported in selected water-quality studies [13], [14], [15], [16], including 83% for WaQuPs, 96.8% for the probabilistic machine-learning approach and 94% for AquaBot. These values are descriptive rather than a controlled benchmark, because the datasets, tasks, feature definitions and validation procedures differ between studies; the comparison provides context but does not establish superiority.

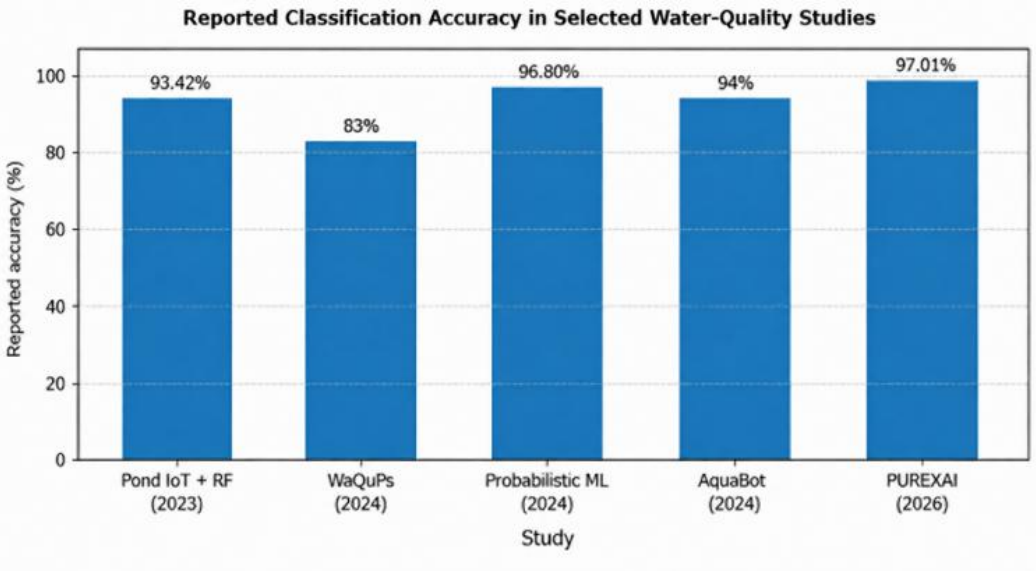


Figure 9 – Reported classification accuracy in selected water-quality studies; values are descriptive and are not a controlled benchmark

Discussion

An important part of interpreting the results is to distinguish feasibility from effectiveness in day-to-day maintenance. Technical feasibility means that the sensing and software workflow can collect observations, generate the defined outputs and communicate alerts under the test conditions. Operational effectiveness would require additional evidence that users respond appropriately, that alerts correspond to independently verified conditions and that the system improves maintenance decisions compared with an established procedure. The current results support the first claim within the documented prototype setup, while the second remains a question for future field evaluation.

The results should also be reproducible from the underlying records. A complete evaluation package would retain the input data or a suitable anonymized dataset, the target definitions, preprocessing configuration, model settings, split boundaries and metric calculations. Such a package would allow another reviewer to check the reported accuracy, F1-scores, MAE and alert latency. Since the supplied report does not contain every original training script or a complete source-code repository, the current document avoids claiming that the full experiment can be reproduced from the report alone. The next project iteration should close this documentation gap by preserving the relevant scripts and test logs.

In summary, the strongest conclusion is appropriately bounded: PUREXAI integrates selected water-quality sensing with maintenance-oriented analytics and multiple user interfaces, and its reported measurements indicate that this workflow operated under the evaluated conditions. Broader claims require independent reference evidence, longer testing and a validation design that separates predictor features from target definitions. Maintaining this distinction makes the project report more credible and provides a clear technical agenda for future development.

The reported results suggest that the components can operate together under the tested prototype conditions. The data collection configuration provides a structured record for analysis, the filter-state model reports strong agreement with the study-defined labels, the remaining-life model provides a numerical estimate, the risk classifier produces an operational category, and the alert workflow responds within the measured average latency. These results support the feasibility of the integrated concept but should not be interpreted as proof of complete safety assessment or universal performance.

The strongest interpretation of the classification result is that the model reproduces the experiment's defined filter-state classes on the held-out data. Because the report identifies a relationship between FHI features and FHI-based classes, additional testing is required to show whether the model generalizes to independent filter-condition evidence. This distinction

matters because a model can achieve a high score by learning the label rule rather than by discovering a physical relationship that transfers to new filters.

The remaining-life result is useful as an initial quantitative measure, but the practical meaning of a 2.8-day MAE depends on the target range and maintenance lead time. A model that is accurate on average may still make large errors for a subset of filters or near a critical replacement point. The next evaluation should therefore report the error distribution and examine performance across relevant operating conditions, provided the original predictions and targets are available. No additional metrics should be added to the present report without the underlying data.

The system's broader value will ultimately depend on whether its outputs help users make better decisions. This question cannot be answered solely by classification metrics. A field evaluation could measure how quickly alerts are investigated, whether the system reduces unnecessary manual checks, how often alerts are confirmed by independent inspection and whether maintenance actions become more timely. Such a study would connect technical performance to operational benefit while preserving the distinction between a prototype monitoring aid and a certified water-quality assessment.

The numerical results should be reported with their evaluation context. The classifier metrics refer to the held-out test partition and the study-defined filter-state classes. The regression MAE refers to the target measured in days. The risk-classifier F1-score refers to its defined risk categories. Alert latency is measured over 100 alert events. Because these measures evaluate different tasks, they should be shown in separate rows and explained individually rather than combined into a single overall score.

The dataset size can be checked from the sampling plan: three nodes multiplied by 1,440 one-minute sampling opportunities per day and ten days gives 43,200 observations. The split percentages correspond to 30,240 training observations, 6,480 validation observations and 6,480 held-out test observations. This consistency check confirms the arithmetic of the stated configuration, although it does not independently verify the completeness or quality of each sensor record.

Comparisons with values reported in other publications should be presented cautiously. Studies may use different sensors, water sources, target definitions, sample sizes, train-test splits and metrics. A percentage from one study cannot be treated as a controlled benchmark against PUREXAI unless the datasets and evaluation procedures are comparable. The current figure is descriptive context and should retain its caption stating that it is not a controlled benchmark.

The discussion should separate what was measured from what is proposed. Reported metrics and the 10-day sampling configuration are project results. Independent laboratory validation, additional sensors, improved uncertainty estimates and a controlled baseline study are future work. This separation keeps the report academically transparent and prevents proposed improvements from being mistaken for features already implemented or evaluated.

The most defensible conclusion is that PUREXAI achieved strong measured performance within its own experimental definition: 97.01% filter-state accuracy, an MAE of 2.8 days, a risk-classifier F1-score of 0.93 and a mean alert latency of 2.1 seconds. Because of the FHI-label relationship noted in Section 4.2.3, the three-tap, 10-day scope and the absence of an independent laboratory reference, broader evaluation is required before cross-system claims can be made. The results indicate technical feasibility under the tested conditions and are not a certification of drinking-water safety.

The results also show why integrated monitoring must be evaluated as a complete operational chain. The sensing stage provides the initial evidence, but that evidence must remain traceable through communication, storage, preprocessing and analysis before it can support an alert. A model metric cannot reveal whether a sensor was disconnected, whether an observation arrived late or whether the dashboard showed a stale value. For this reason, future evaluation should combine model-level measures with data-completeness checks, device-health records, alert-delivery outcomes and documented inspection results. This broader evaluation would make it possible to understand not only whether a prediction matches its target, but also whether the system can provide timely and interpretable information during routine operation.

The project’s practical contribution is therefore best expressed as a documented prototype workflow with measurable components and explicit boundaries. The reported metrics provide a starting point for further experiments, while the identified limitations define the next validation tasks. A stronger subsequent study should use independent reference measurements, a clearly defined filter-condition target, a leakage-controlled data pipeline and a longer observation period across multiple sites. It should also record maintenance actions and evaluate whether alerts lead to useful follow-up. Until such evidence is available, PUREXAI should be used as an operational monitoring and decision-support prototype, not as a replacement for certified water testing or established maintenance responsibilities.

REFERENCES

[1] Espressif Systems, "ESP32 Wi-Fi & Bluetooth SoC," Espressif Systems. [Online]. Available: https://www.espressif.com/en/products/socs/esp32

[2] I. Essamlali, H. Nhaila, and M. El Khaili, "Advances in machine learning and IoT for water quality monitoring: A comprehensive review," *Heliyon*, vol. 10, no. 6, e27920, 2024, doi: 10.1016/j.heliyon.2024.e27920.

[3] K. Pandey, P. Shukla, S. Bhakar, H. Tekchandani, and R. Mittal, "A systematic review on machine learning, deep learning and IoT for water quality prediction," *Discover Sustainability*, vol. 7, art. 1279, 2026, doi: 10.1007/s43621-026-03756-y.

[4] P. Dutta and S. Sarma, "Smart Water Management: Role of IoT, AI, and Machine Learning in Water Quality Monitoring," *Water Environment Research*, vol. 98, no. 2, e70249, 2026, doi: 10.1002/wer.70249.

[5] G. Dharmarathne, A. M. S. R. Abekoon, M. Bogahawaththa, J. Alawatugoda, and D. P. P. Meddage, "A review of machine learning and internet-of-things on the water quality assessment: Methods, applications and future trends," *Results in Engineering*, vol. 26, art. 105182, 2025, doi: 10.1016/j.rineng.2025.105182.

[6] World Health Organization, *Guidelines for Drinking-water Quality: Fourth Edition Incorporating the First and Second Addenda*. Geneva, Switzerland: World Health Organization, 2022.

[7] Espressif Systems, *ESP32 Series Datasheet and Technical Documentation*, Espressif Systems.

[8] P. Jayaraman, K. K. Nagarajan, P. Partheeban, and V. Krishnamurthy, "Critical review on water quality analysis using IoT and machine learning models," *Journal of Information, Measurement and Decision Sciences*, vol. 4, no. 1, art. 100210, 2024, doi: 10.1016/j.jjimei.2023.100210.

[9] Z. Li, W. Ma, D. Zhong, J. Ma, Q. Zhang, Y. Yuan, X. Liu, X. Wang, and K. Zou, "Applications of machine learning in drinking water quality management: A critical review on water distribution system," *Journal of Cleaner Production*, vol. 481, art. 144171, 2024, doi: 10.1016/j.jclepro.2024.144171.

[10] I. A. Adeleke, N. I. Nwulu, and O. A. Ogbolumani, "A hybrid machine learning and embedded IoT-based water quality monitoring system," *Internet of Things*, vol. 22, art. no. 100774, 2023, doi: 10.1016/j.iot.2023.100774.

[11] B. Ngwenya, T. Paepae, and P. N. Bokoro, "Monitoring ambient water quality using machine learning and IoT: A review and recommendations for advancing SDG indicator 6.3.2," *Journal of Water Process Engineering*, vol. 73, art. 107664, 2025, doi: 10.1016/j.jwpe.2025.107664.

[12] S. O. Olatinwo and T. H. Joubert, “Resource Allocation Optimization in IoT-Enabled Water Quality Monitoring Systems,” *Sensors*, vol. 23, no. 21, art. 8963, 2023, doi: 10.3390/s23218963.

[13] F. Firdiani, S. Mandala, Adiwijaya, and A. H. Abdullah, “WaQuPs: A ROS-Integrated Ensemble Learning Model for Precise Water Quality Prediction,” *Applied Sciences*, vol. 14, no. 1, art. 262, 2024, doi: 10.3390/app14010262.

[14] M. M. Islam, M. A. Kashem, S. A. Alyami, and M. A. Moni, “Monitoring water quality metrics of ponds with IoT sensors and machine learning to predict fish species survival,” *Microprocessors and Microsystems*, vol. 102, art. 104930, 2023, doi: 10.1016/j.micpro.2023.104930.

[15] “Water quality level estimation using IoT sensors and probabilistic machine learning model,” *Hydrology Research*, vol. 55, no. 7, pp. 775–789, 2024, doi: 10.2166/nh.2024.048.

[16] M. M. Hemal, A. Rahman, Nurjahan, F. Islam, S. Ahmed, M. S. Kaiser, and M. R. Ahmed, “An Integrated Smart Pond Water Quality Monitoring and Fish Farming Recommendation Aquabot System,” *Sensors*, vol. 24, no. 11, art. 3682, 2024, doi: 10.3390/s24113682.